Transform and Conquer

Chapter 6: Transform and Conquer

This group of techniques solves a problem by a *transformation* to

- **a simpler/more convenient instance of the same problem (*instance simplification*)**
- **a different representation of the same instance (*representation change*)**
- **a different problem for which an algorithm is already available (*problem reduction*)**

Objectives

To learn and appreciate the following concepts:

- ✓Presorting
- ✓Balanced Search Trees
- ✓Heaps and Heapsort
- ✓Problem Reduction

6.1 Instance simplification - Presorting

Solve a problem's instance by transforming it into another simpler/easier instance of the same problem

Presorting

Many problems involving lists are easier when list is sorted, e.g

- searching
- computing the median (selection problem)
- checking if all elements are distinct (element uniqueness)

Also:

- Topological sorting helps solving some problems for dags.
- Presorting is used in many geometric algorithms.

Presorting

- **EXAMPLE 1:** Checking element uniqueness in an array
- The brute-force algorithm compared pairs of the array's elements until either two equal elements were found or no more pairs were left.
- Its worst-case efficiency was in (n^2).
- we can sort the array first and then check only its consecutive elements: if the array has equal elements, a pair of them must be next to each other, and vice versa.

Presorting

- The algorithm for presort Element Uniqueness is:

ALGORITHM *PresortElementUniqueness*$(A[0..n-1])$
//Solves the element uniqueness problem by sorting the array first
//Input: An array $A[0..n-1]$ of orderable elements
//Output: Returns "true" if A has no equal elements, "false" otherwise
sort the array A
for $i \leftarrow 0$ **to** $n-2$ **do**
    if $A[i] = A[i+1]$ **return false**
return true

- The running time of this algorithm is the sum of the time spent on sorting and the time spent on checking consecutive elements. Since the former requires at least n log n comparisons and the latter needs no more than n − 1 comparisons, it is the sorting part that will determine the overall efficiency of the algorithm. So, if we use a quadratic sorting algorithm here, the entire algorithm will not be more efficient than the brute-force one. But if we use a good sorting algorithm, such as mergesort, with worst-case efficiency in (n log n), the worst-case efficiency of the entire presorting-based algorithm will be also in (n log n):

$$\mathbf{T(n) = T_{sort}(n) + T_{scan}(n) \in (n \log n) + (n) = (n \log n).}$$

Presorting

- **EXAMPLE 2:** Computing a mode A mode is a value that occurs most often in a given list of numbers.
- For example, for 5, 1, 5, 7, 6, 5, 7, the mode is 5. (If several different values occur most often, any of them can be considered a mode.)
- The brute-force approach to computing a mode would scan the list and compute the frequencies of all its distinct values, then find the value with the largest frequency.

Presorting

- we can store the values already encountered, along with their frequencies, in a separate list. On each iteration, the ith element of the original list is compared with the values already encountered by traversing this auxiliary list. If a matching value is found, its frequency is incremented; otherwise, the current element is added to the list of distinct values seen so far with a frequency of 1. It is not difficult to see that the worst-case input for this algorithm is a list with no equal elements. For such a list, its ith element is compared with i − 1 elements of the auxiliary list of distinct values seen so far before being added to the list with a frequency of 1. As a result, the worst-case number of comparisons made by this algorithm in creating the frequency list is:

$$C(n) = \sum_{i=1}^{n} (i-1) = 0 + 1 + \cdots + (n-1) = \frac{(n-1)n}{2} \in \Theta(n^2).$$

- The additional n − 1 comparisons needed to find the largest frequency in the auxiliary list do not change the quadratic worst-case efficiency class of the algorithm.

Presorting

- EXAMPLE 3: Searching problem Consider the problem of searching for a given value v in a given array of n sortable items.
- The brute-force solution here is sequential search, which needs n comparisons in the worst case.
- If the array is sorted first, we can then apply binary search, which requires only log n + 1 comparisons in the worst case. Assuming the most efficient n log n sort, the total running time of such a searching algorithm in the worst case will be:

$$T(n) = T_{sort}(n) + T_{search}(n) = \Theta(n \log n) + \Theta(\log n) = \Theta(n \log n),$$

- which is inferior to sequential search. The same will also be true for the average case efficiency. Of course, if we are to search in the same list more than once, the time spent on sorting might well be justified.

Presorting - TRY

1. Consider the problem of finding the distance between the two closest numbers in an array of n numbers. (The distance between two numbers x and y is computed as $|x - y|$.)

- a. Design a presorting-based algorithm for solving this problem and determine its efficiency class.
- b. Compare the efficiency of this algorithm with that of the brute-force algorithm.

Representation Change

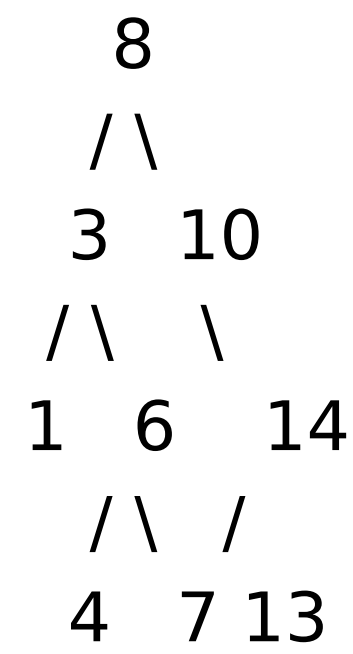
- A different representation of the same instance

Introduction to Binary Search Trees

- Definition: A Binary Search Tree (BST) is a node-based data structure where each node has up to two children, and the left child is less than the parent node, while the right child is greater.
- Importance: Efficient for search, insertion, and deletion operations.

Structure of a BST

- Node Structure: Each node contains a value, a reference to the left child, and a reference to the right child.
- Example:



Properties of BST

- Ordering Property: Left subtree values < node value < right subtree values.
- No Duplicates: Typically, BSTs do not contain duplicate values.

Searching in a BST

Algorithm:

1. Start at the root.
2. Compare the target value with the current node's value.
3. Move left if the target is smaller, or right if it's larger.
4. Repeat until the value is found or the subtree is empty.

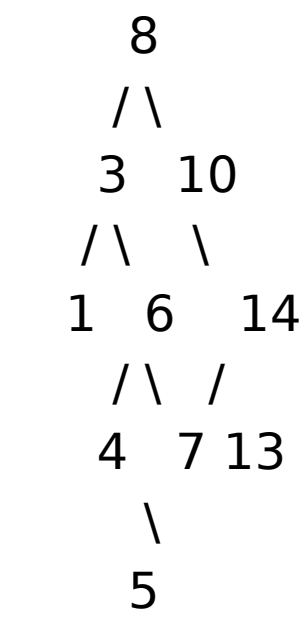
Example: Searching for 7 in the example BST.

Insertion in a BST

Algorithm:

1. Start at the root.
2. Compare the value to be inserted with the current node's value.
3. Move left if the value is smaller, or right if it's larger.
4. Insert the new node at the position where the search would end.

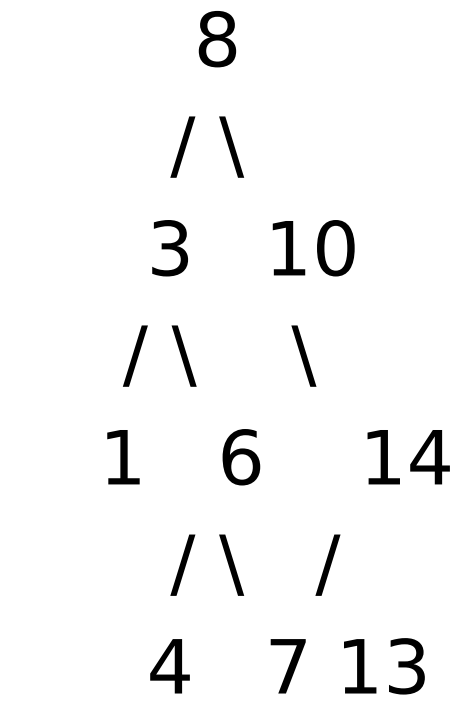
Example: Inserting 5 into the example BST.



Algorithm:

Case 1: Node with No Children (Leaf Node)

Example: Deleting node 1 from the BST.



Explanation: Node 1 is a leaf node. Simply remove it.

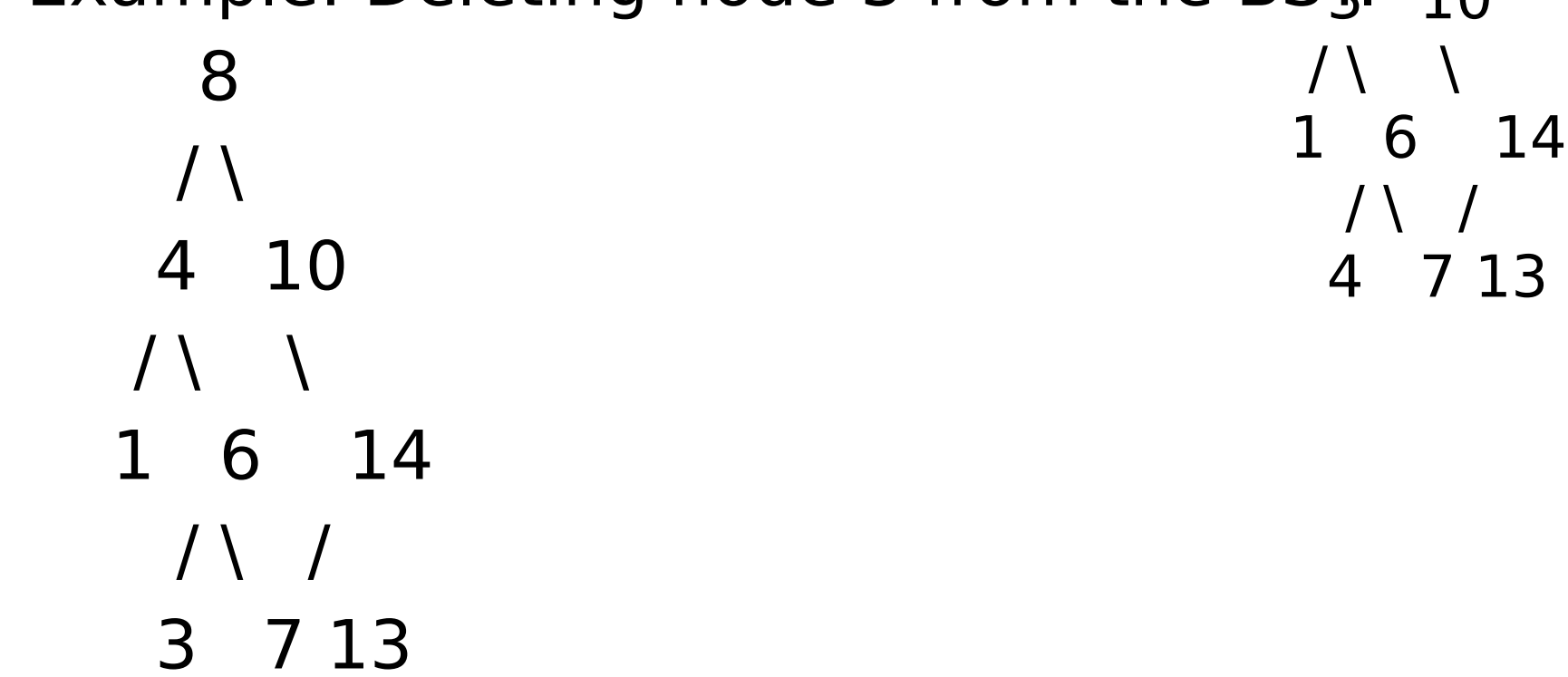
Case 2: Node with One Child

Example: Deleting node 14 from the BST.

Explanation: Node 14 has one child (13).
Remove node 14 and link 13 directly to 10.

Case 3: Node with Two Children

Example: Deleting node 3 from the BST.



Explanation: Node 3 has two children (1 and 6). Find the in-order successor (4), swap values, and delete the successor.

Applications of BST

- Usage: Databases, search engines, and memory management.
- Advantages: Efficient search, insertion, and deletion operations.

- **AVL tree**
- A **red-black tree**
- If an insertion or deletion of a new node creates a tree with a violated balance requirement, the tree is restructured by one of a family of special transformations called **rotations** that restore the balance required.

- **B-trees**.
  - **2-3 trees, 2-3-4 tree**

Balanced binary tree

- The disadvantage of a binary search tree is that its height can be as large as N-1
- This means that the time needed to perform insertion and deletion and many other operations can be O(N) in the worst case
- We want a tree with small height
- A binary tree with N node has height at least  (log N)
- Thus, our goal is to keep the height of a binary search tree O(log N)
- Such trees are called balanced binary search trees. Examples are AVL tree, red-black tree.

AVL-Trees

(Adelson-Velskii and Landis-Trees)

AVL tree

AVL trees were invented in 1962 by two Russian scientists, G. M. Adelson-Velsky and E. M. Landis [Ade62], after whom this data structure is named.

Height of a node

- The height of a leaf is 0. The height of an empty tree is -1.
- The height of an internal node is the maximum height of its children plus 1

AVL tree

- An AVL tree is a binary search tree in which
  - for *every* node in the tree, the height of the left and right subtrees differ by at most 1.

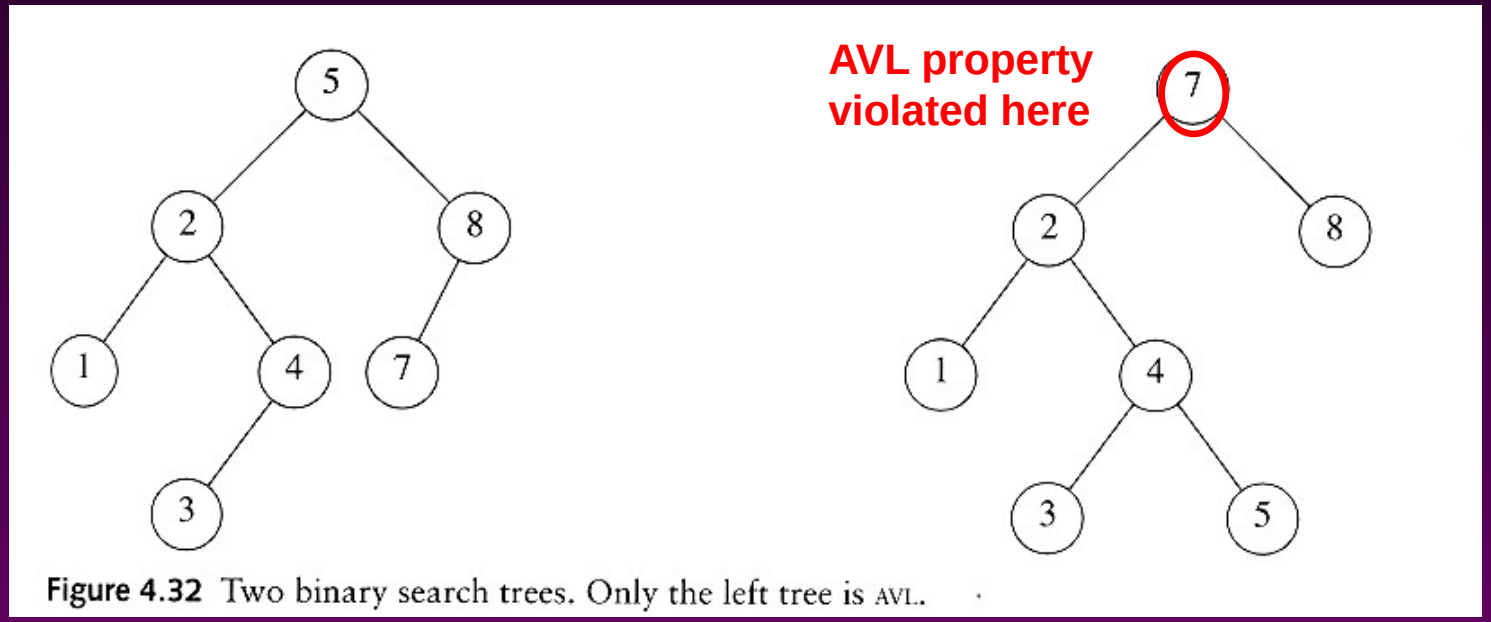


Figure 4.32 Two binary search trees. Only the left tree is AVL.

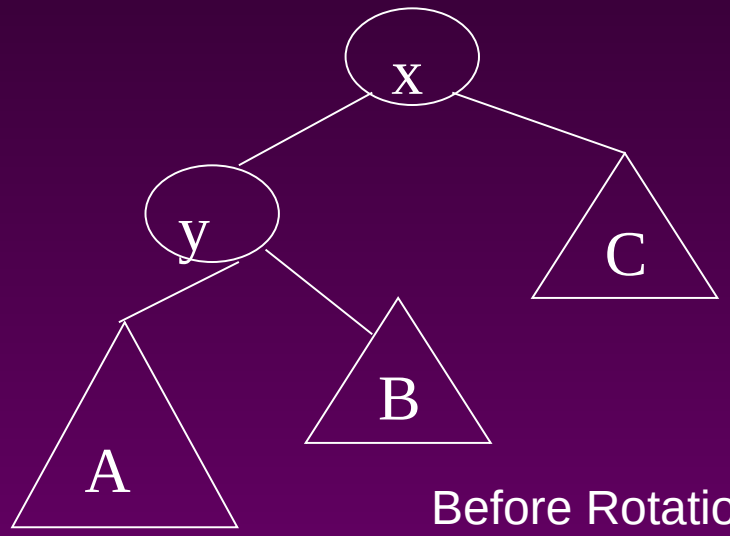
AVL tree

- Let x be the root of an AVL tree of height h
- Let S(h) denote the minimum number of nodes in an AVL tree of height h.
- S(h)=S(h-1)+S(h-2)+1
- Many operations (searching, insertion, deletion) on an AVL tree will take O(log N) time.

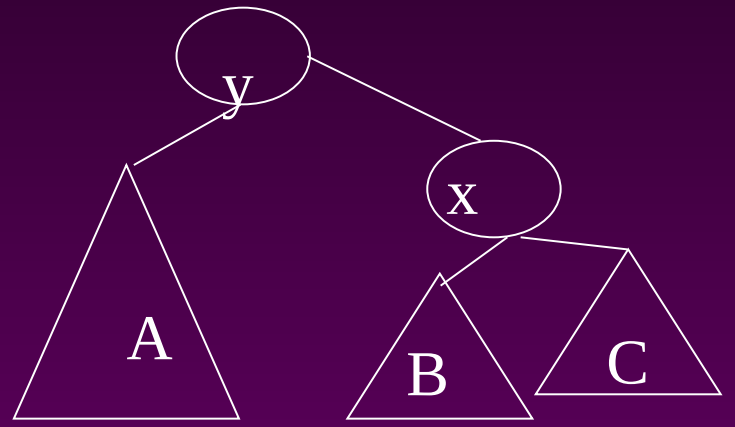
Rotations

- When the tree structure changes (e.g., insertion or deletion), we need to transform the tree to restore the AVL tree property.
- This is done using single rotations or double rotations.

e.g. Single Rotation



Before Rotation



After Rotation

Rotations

- Since an insertion/deletion involves adding/deleting a single node, this can only increase/decrease the height of some subtree by 1
- Thus, if the AVL tree property is violated at a node x, it means that the heights of left(x) and right(x) differ by exactly 2.
- Rotations will be applied to x to restore the AVL tree property.

Insertion

- First, insert the new key as a new leaf just as in ordinary binary search tree
- Then trace the path from the new leaf towards the root. For each node x encountered, check if heights of left(x) and right(x) differ by at most 1.
- If yes, proceed to parent(x). If not, restructure by doing either a single rotation or a double rotation
- For insertion, once we perform a rotation at a node x, we don’t need to perform any rotation at any ancestor of x.

Insertion

- Let x be the node at which left(x) and right(x) differ by more than 1
- A violation may occur in four cases:

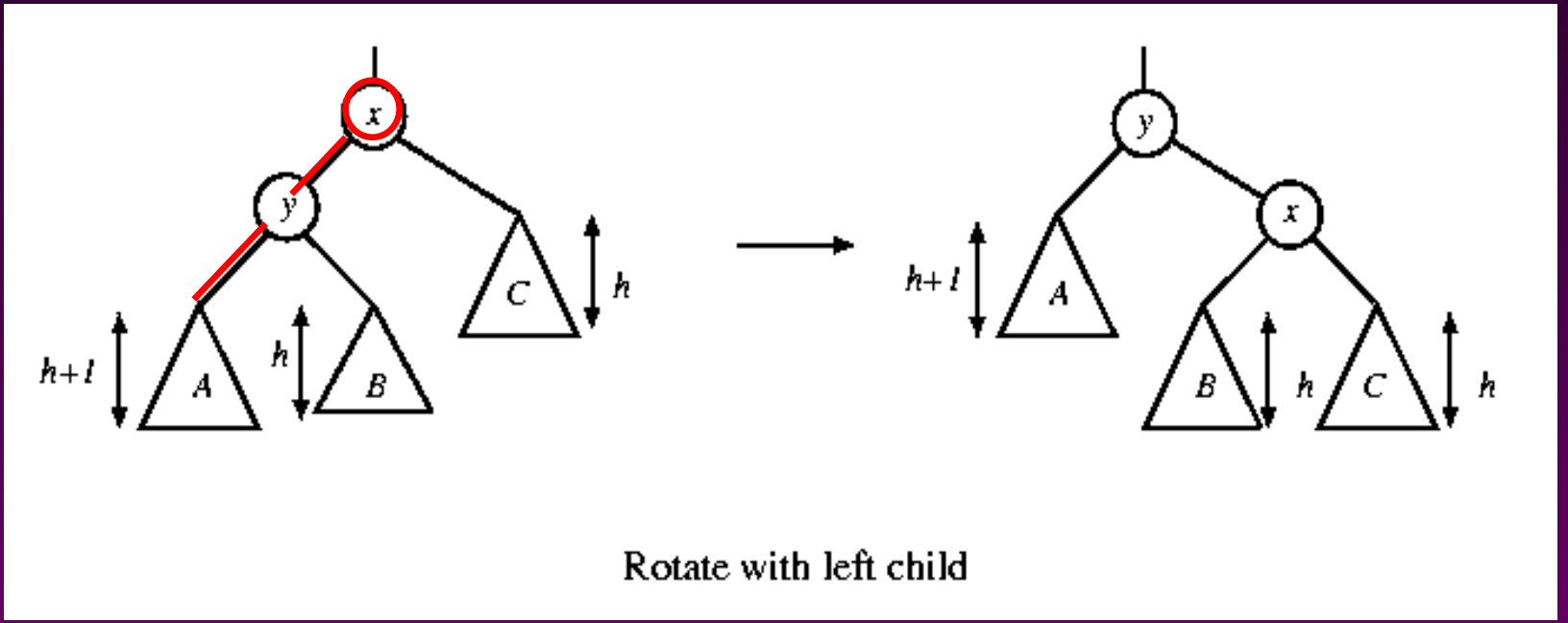
1. An insertion into the left subtree of the left child of x
2. An insertion into the right subtree of the left child of x
3. An insertion into the left subtree of the right child of x.
4. An insertion into the right subtree of the right child of x.

Single rotation

The new key is inserted in the subtree A.

The AVL-property is violated at x

- height of left(x) is h+2
- height of right(x) is h.

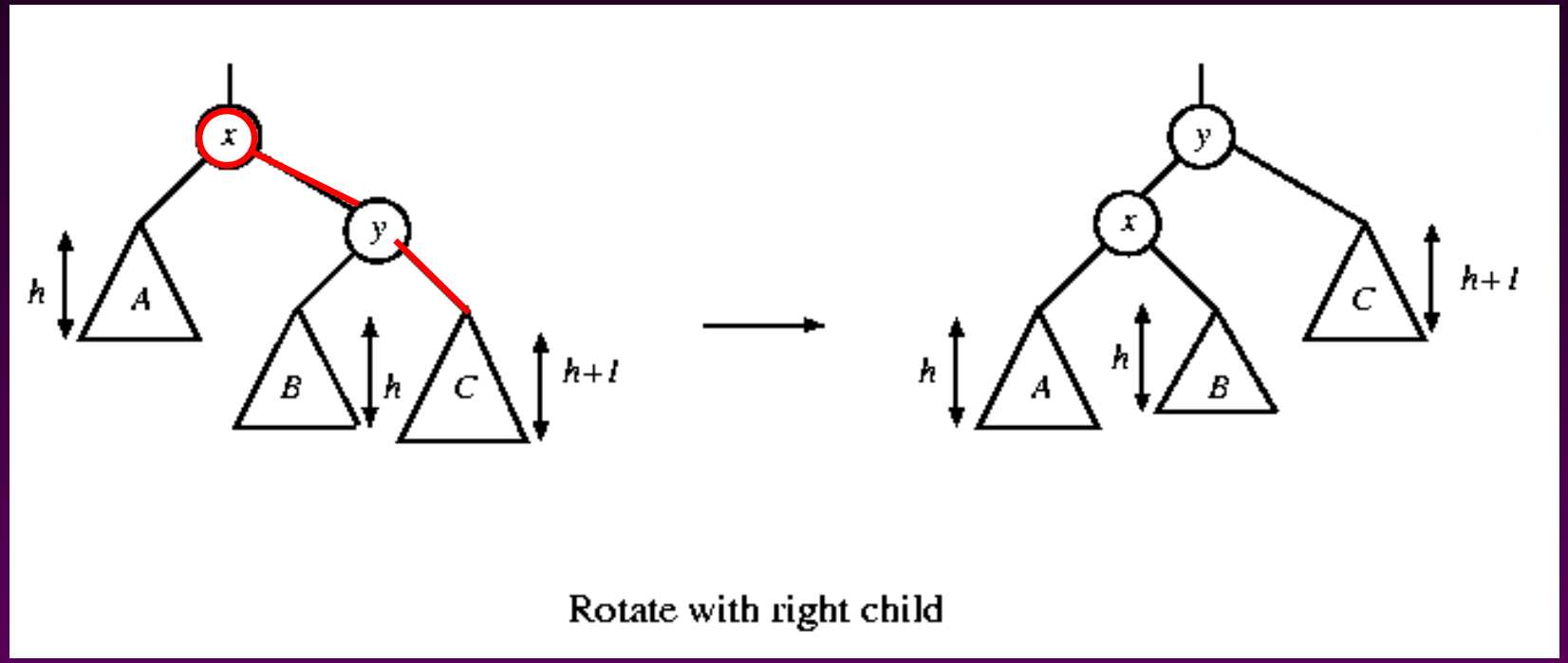


Rotate with left child

Single rotation

The new key is inserted in the subtree C.

The AVL-property is violated at x.

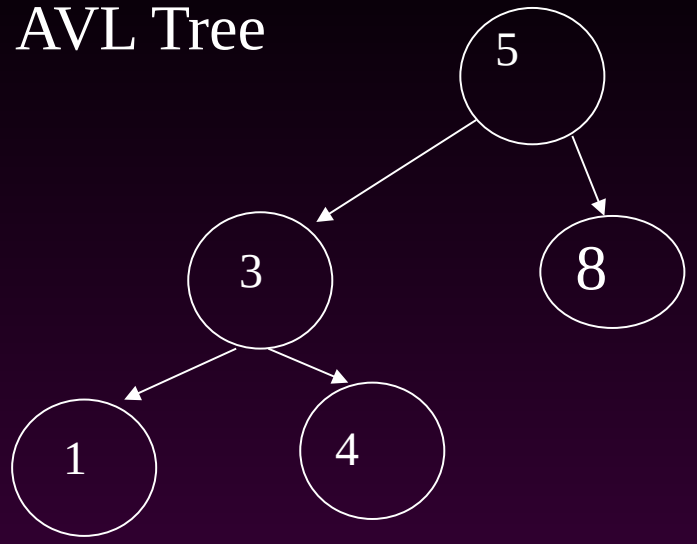


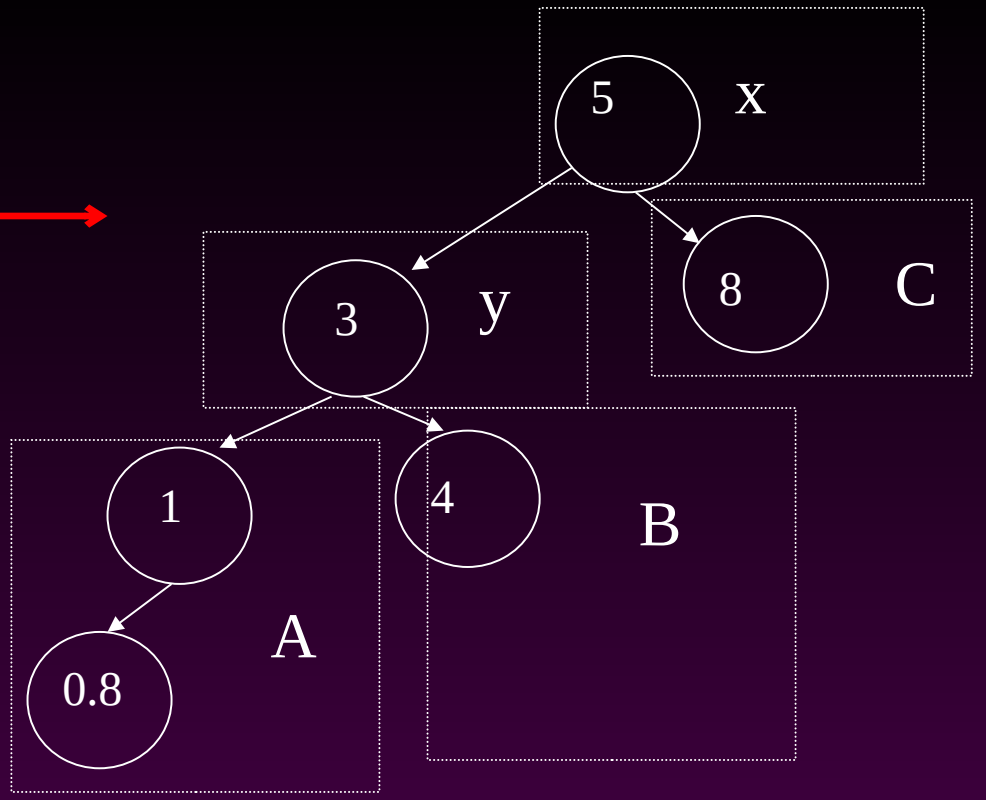
Rotate with right child

Single rotation takes O(1) time.

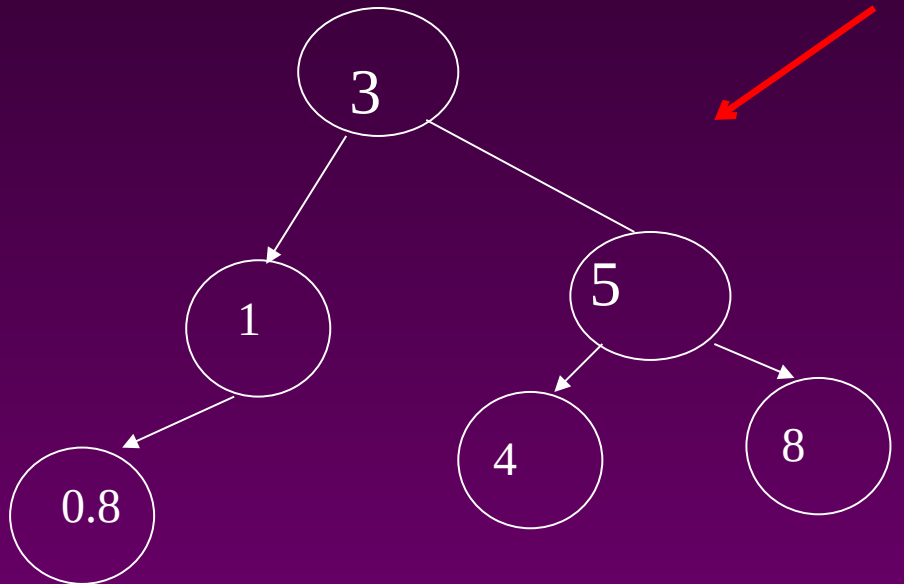
Insertion takes O(log N) time.

AVL Tree





Insert 0.8



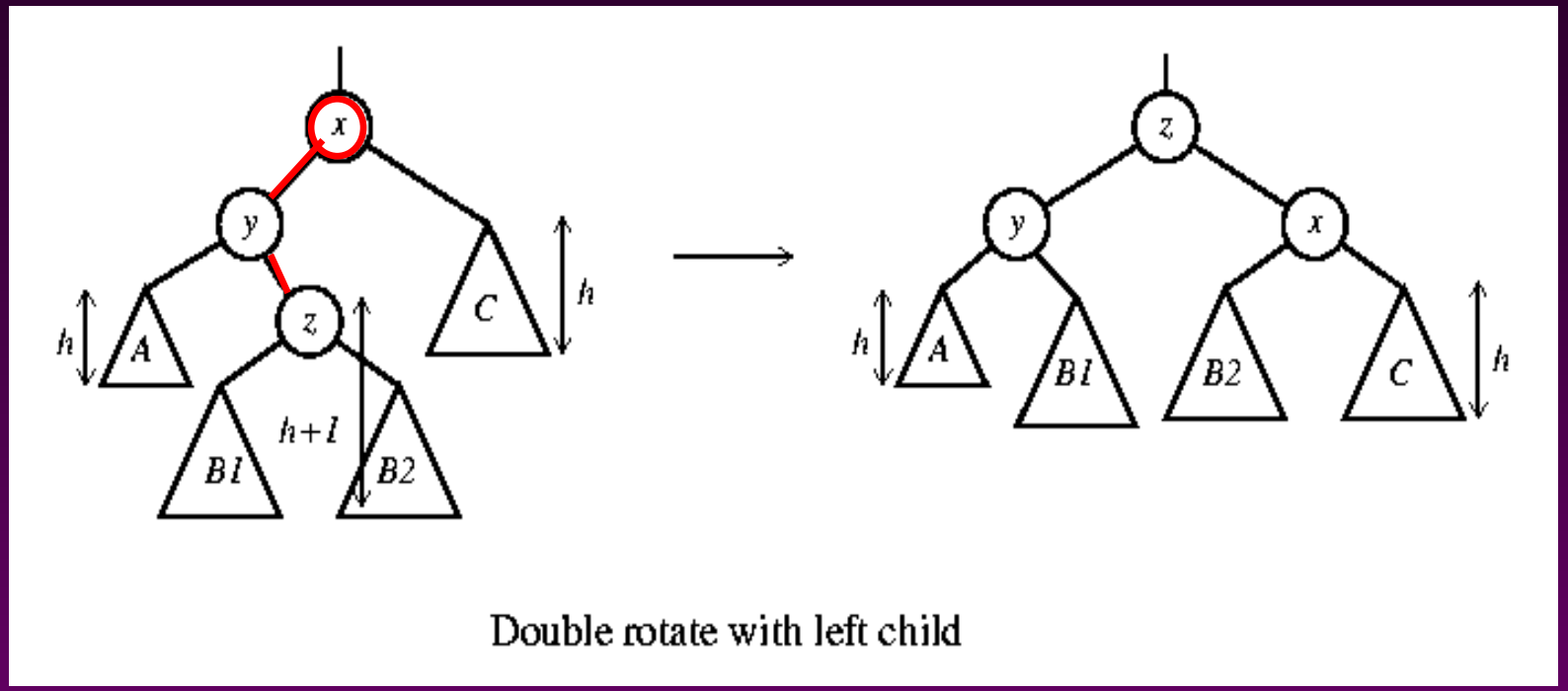
After rotation

Double rotation

The new key is inserted in the subtree B1 or B2.

The AVL-property is violated at x.

x-y-z forms a zig-zag shape



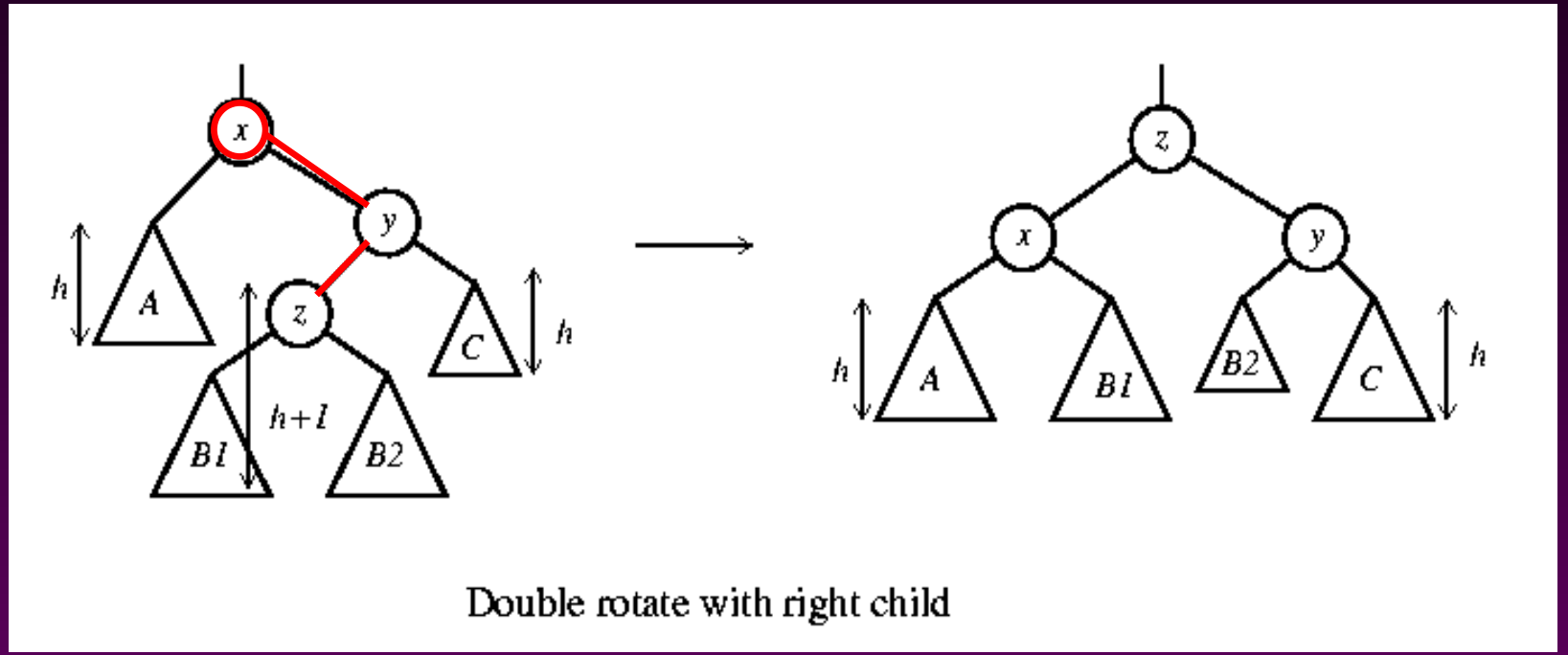
Double rotate with left child

also called left-right rotate

Double rotation

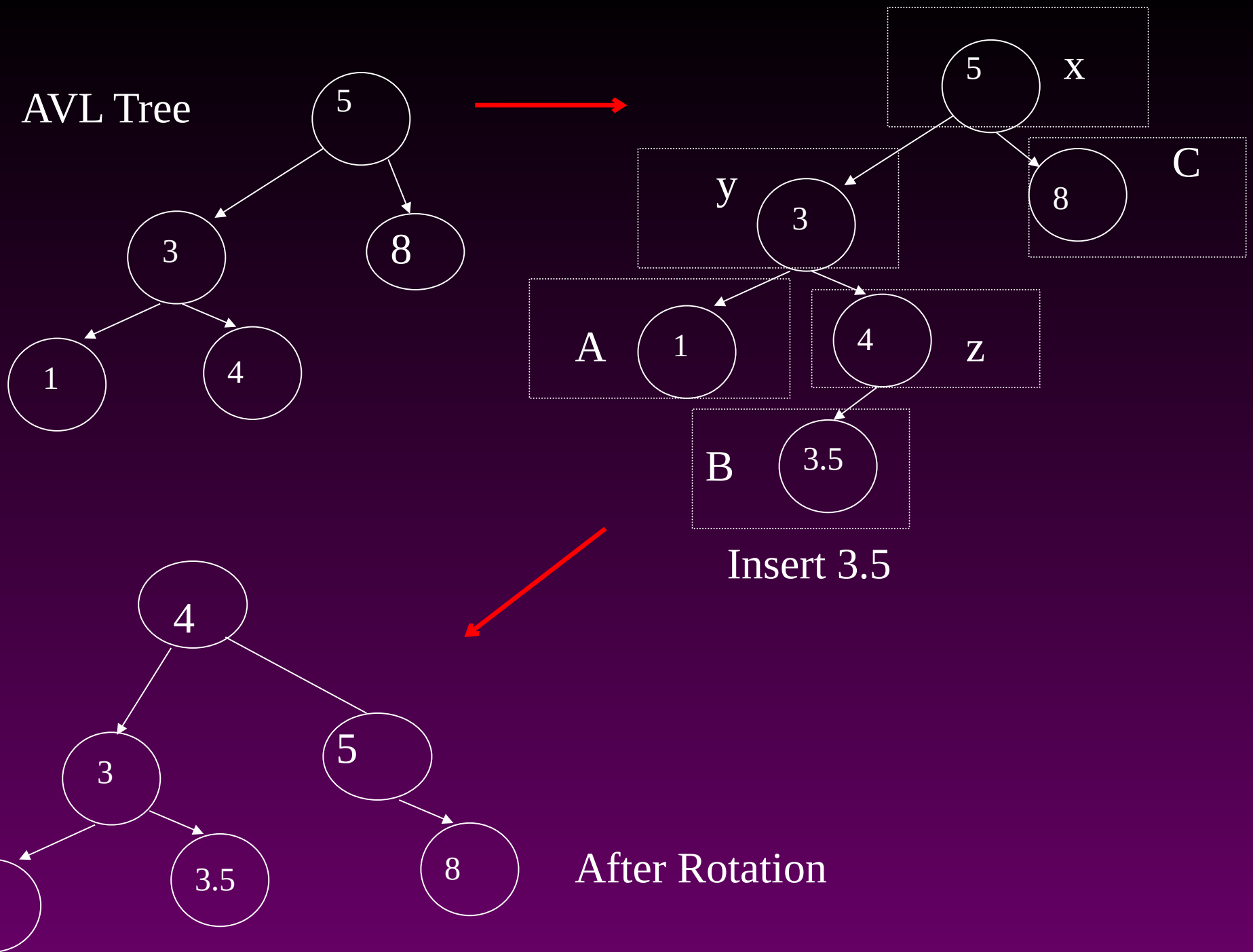
The new key is inserted in the subtree B1 or B2.

The AVL-property is violated at x.



Double rotate with right child

also called right-left rotate


AVL Tree
5
3
8
1
4
5
x
y
3
C
8
A
1
4
z
B
3.5
Insert 3.5
4
3
5
1
3.5
8
After Rotation

An Extended Example

Insert 3,2,1,4,5,6,7, 16,15,14

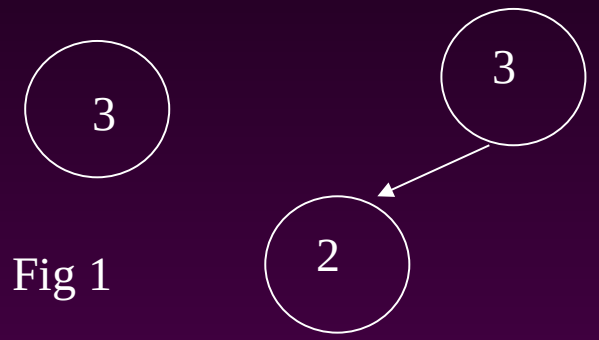


Fig 1

Fig 2

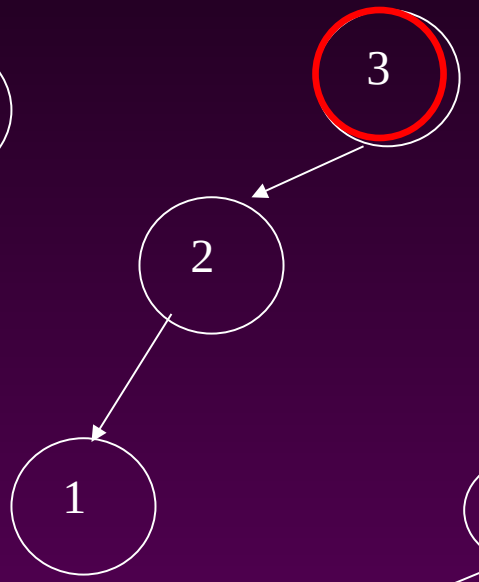


Fig 3

Single rotation

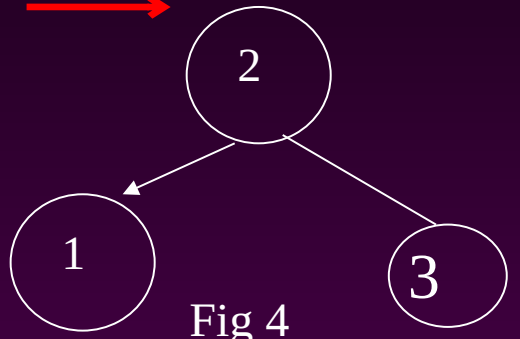


Fig 4

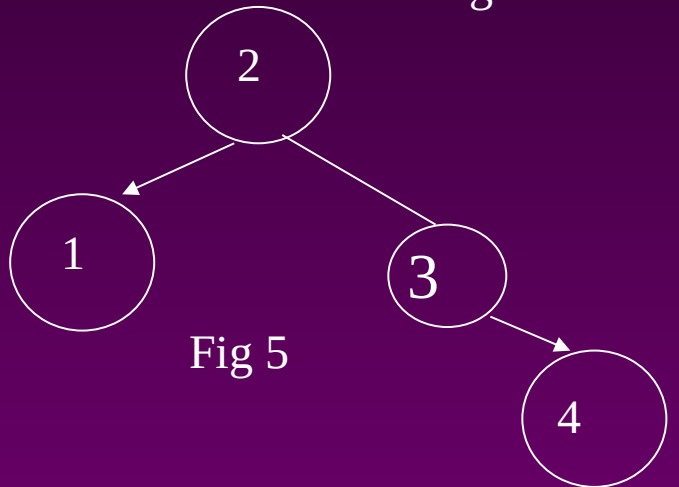


Fig 5

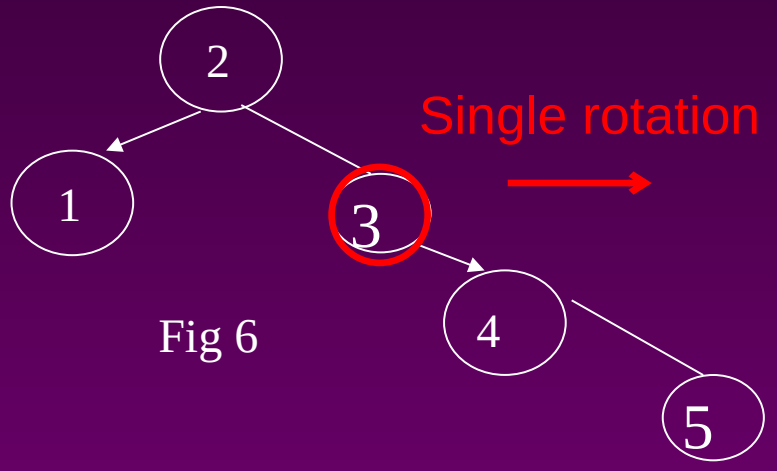


Fig 6

Single rotation

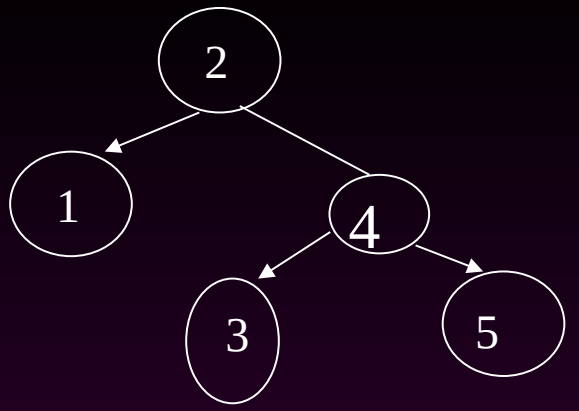
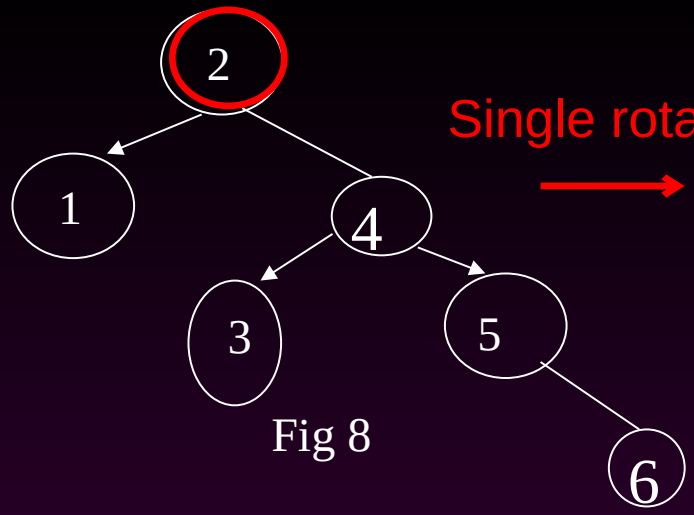


Fig 7



Single rotation

Fig 8

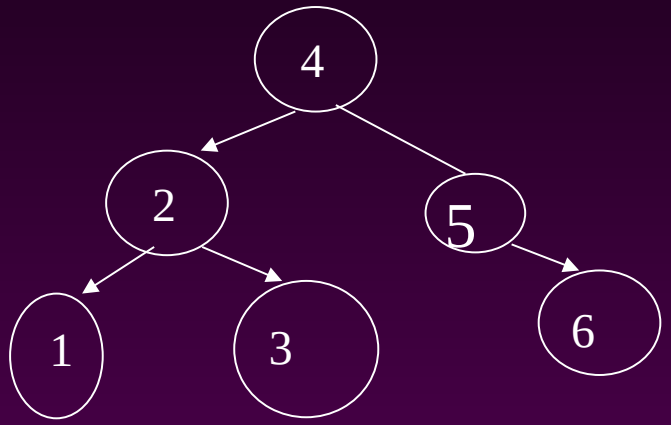
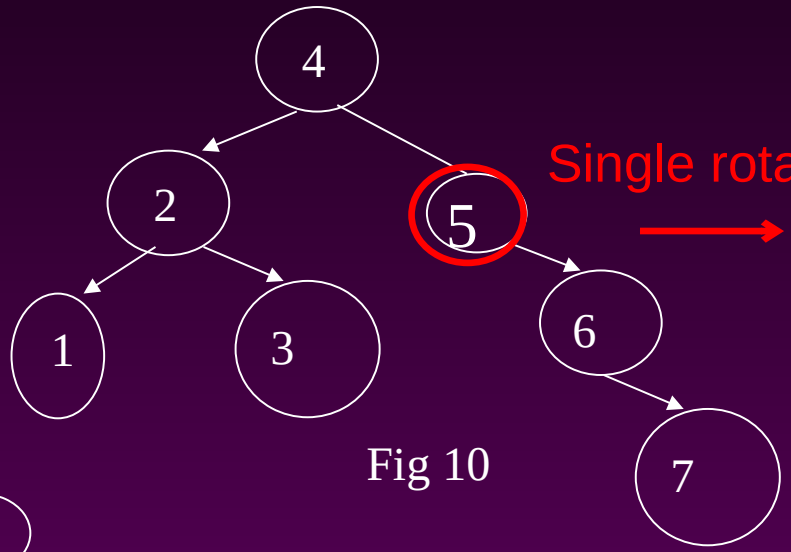


Fig 9



Single rotation

Fig 10

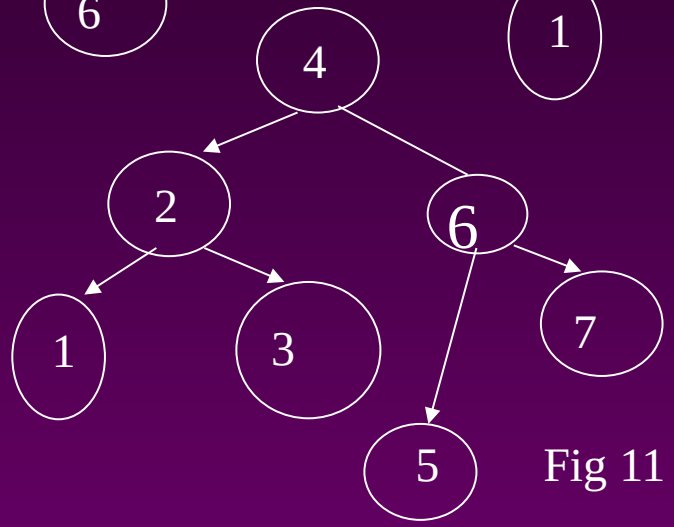


Fig 11

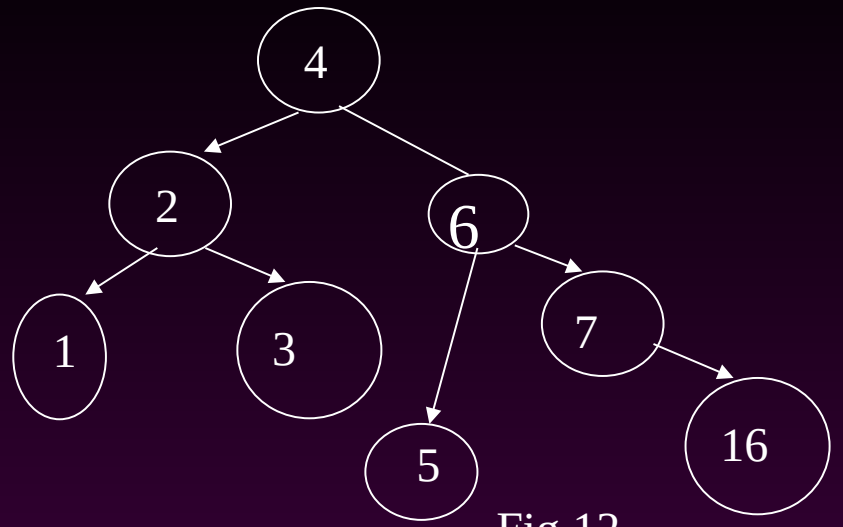


Fig 12

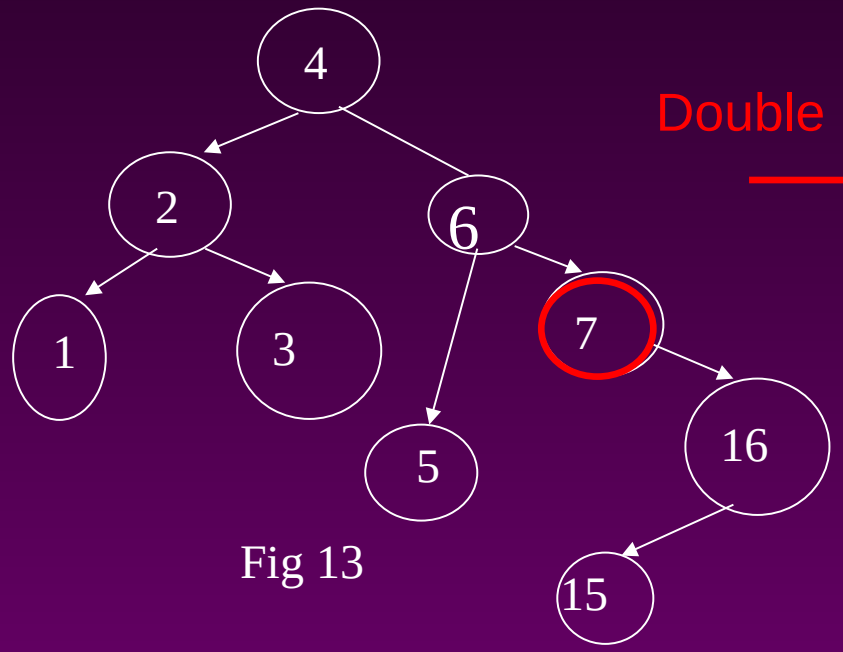


Fig 13

Double rotation

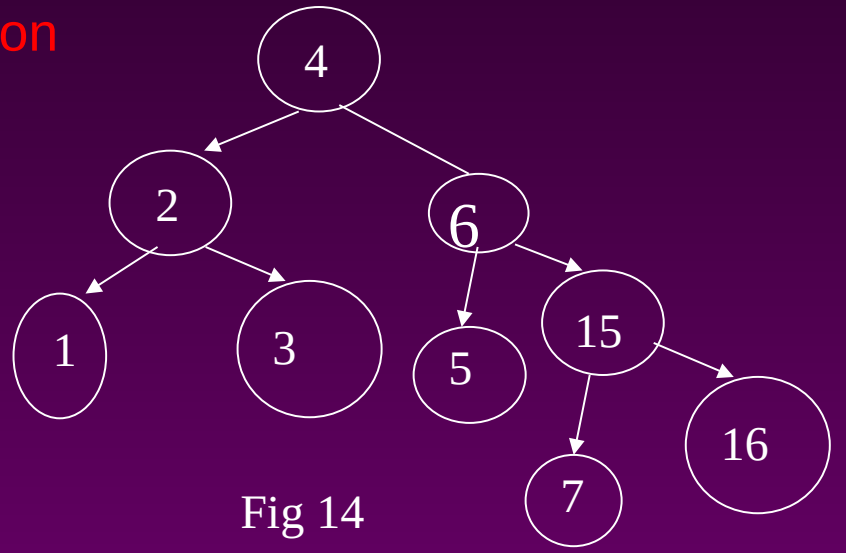


Fig 14

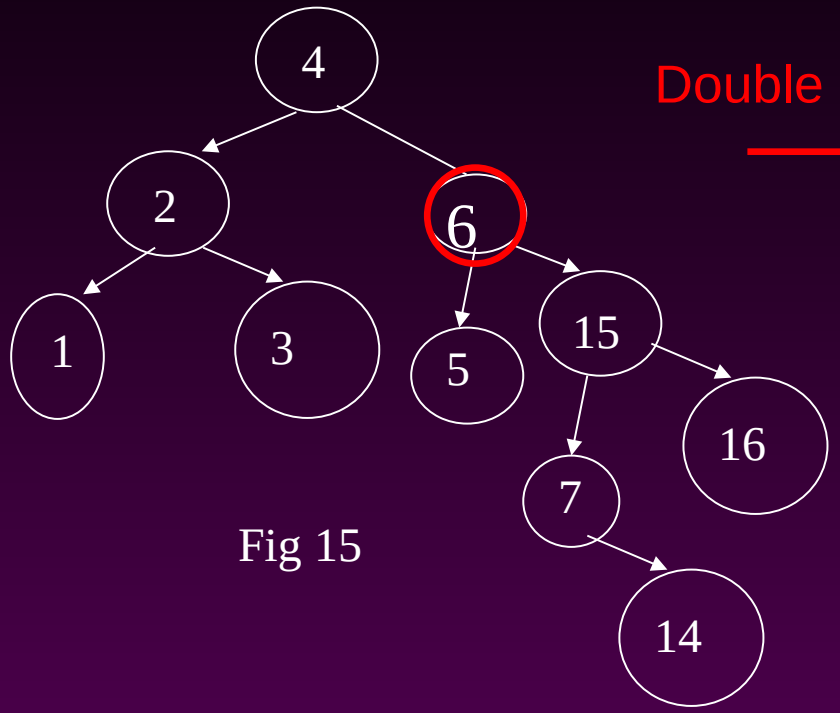


Fig 15

Double rotation

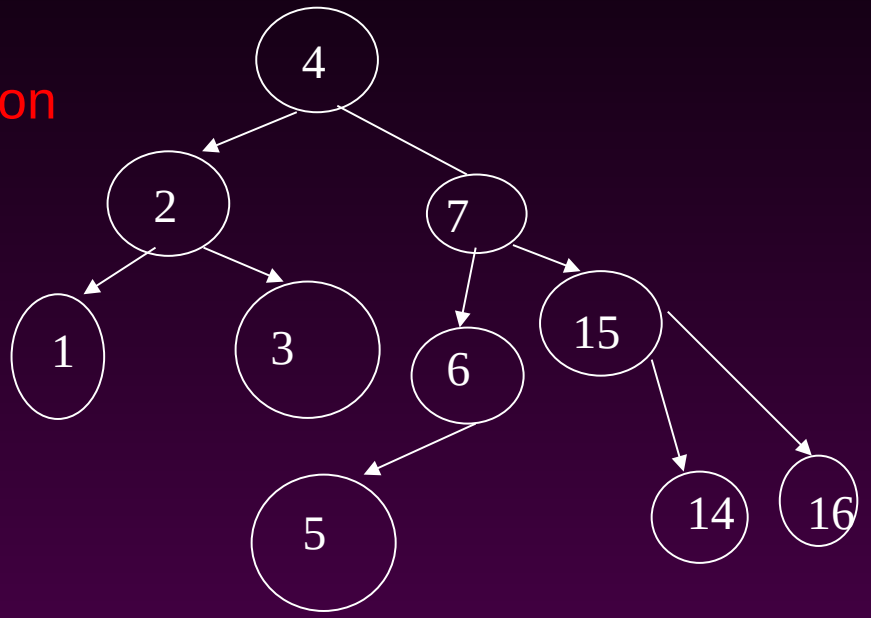
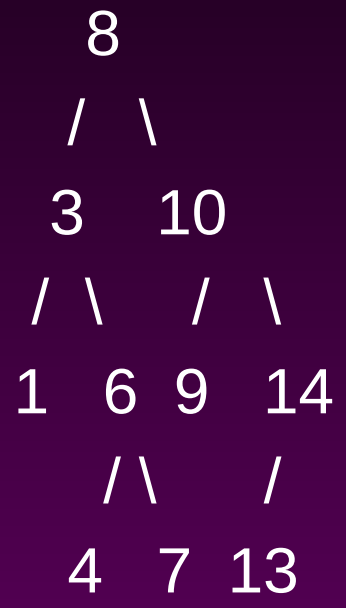


Fig 16

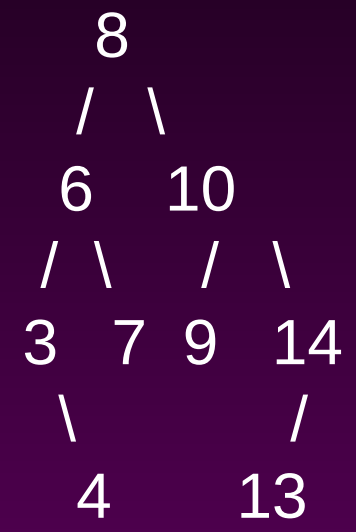
Algorithm:

Case 1: Node with No Children (Leaf Node)

Example: Deleting node 1 .



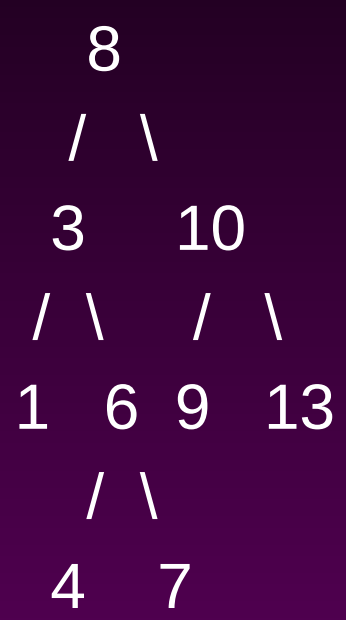
Single Rotation

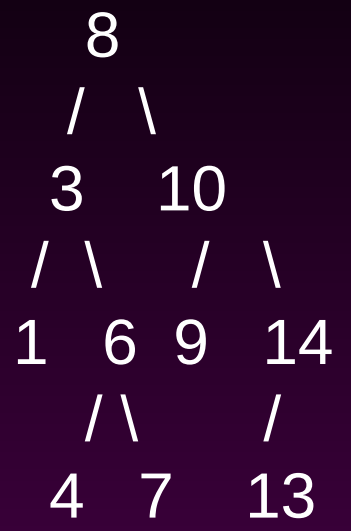


Explanation: Node 1 is a leaf node. Simply remove it. Check for AVL tree property violations

Case 2: Node with One Child

Example: Deleting node 14

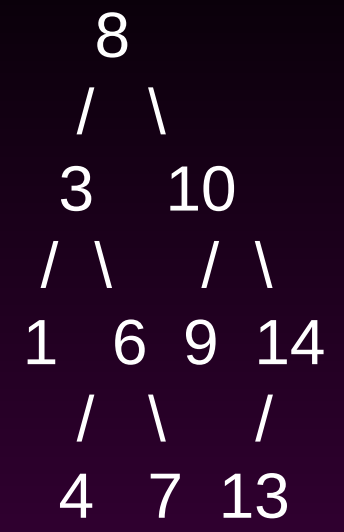




Explanation: Node 14 has one child (13). Remove node 14 and link 13 directly to 10. Check for AVL tree property violations

Case 3: Node with Two Children

Example: Deleting node 3 .



Explanation: Node 3 has two children (1 and 6). Find the in-order successor (4), swap values, and delete the successor.. Check for AVL tree property violations

Create Avl tree with following keys

30, 40, 25, 20, 5, 50, 45, 48, 60, 70, 65, 80, 90, 100, 10, 2

After construction delete the keys 60, 50, 45, 30

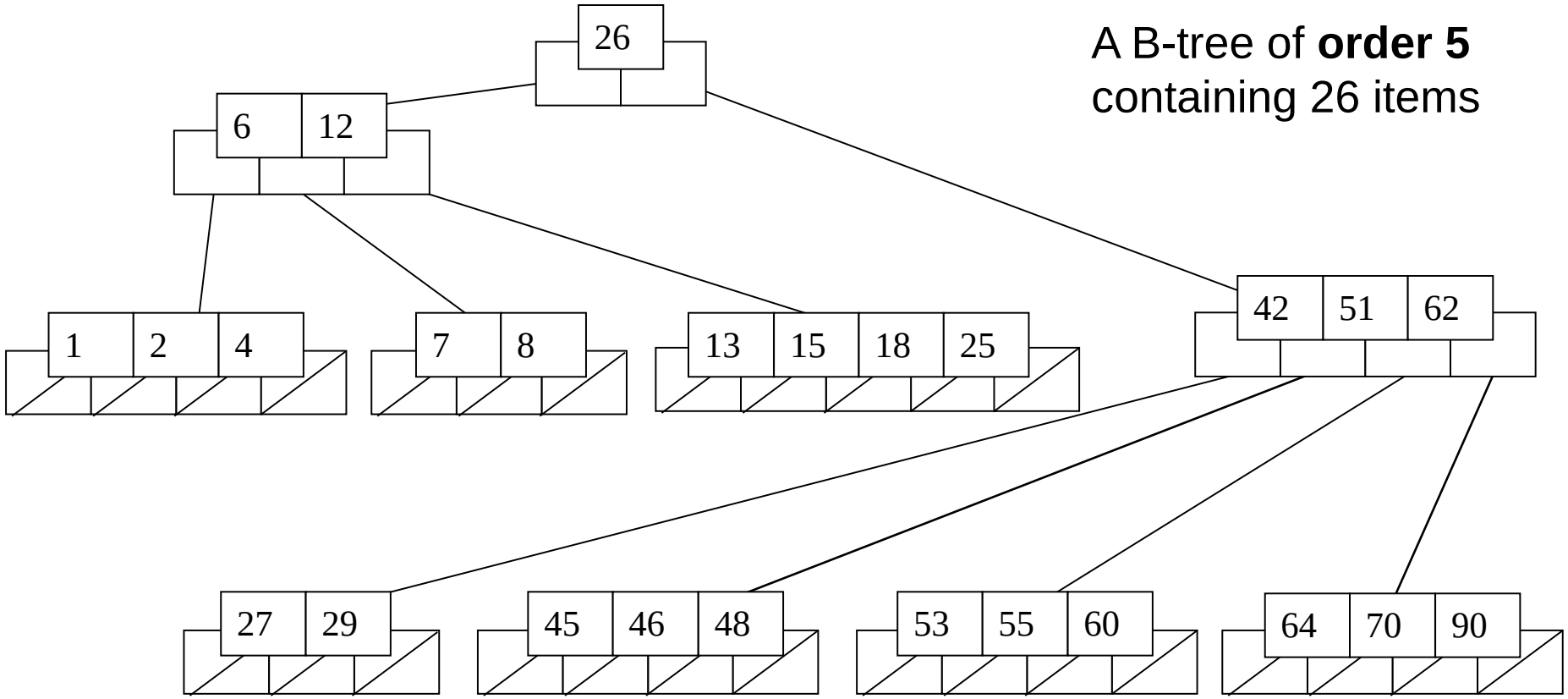
For Visualization:

https://www.cs.usfca.edu/~galles/visualization/AVLtree.html

Definition of a B-tree

- A B-tree of order *m* is an *m*-way tree (i.e., a tree where each node may have up to *m* children) in which:
  1. the number of keys in each non-leaf node is one less than the number of its children.
  2. all leaves are on the same level
  3. all non-leaf nodes except the root have at least $m / 2$ children
  4. the root is either a leaf node, or it has from two to *m* children
  5. a leaf node contains no more than $m - 1$ keys

An example B-Tree



A B-tree of **order 5** containing 26 items

Note that all the leaves are at the same level

Analysis of B-Trees

- The maximum number of items in a B-tree of order m and height h:

  root $m - 1$

  level 1 $m(m - 1)$

  level 2 $m^2(m - 1)$

  . . .

  level h $m^h(m - 1)$

- So, the total number of items is

$$(1 + m + m^2 + m^3 + \ldots + m^h)(m - 1) =$$

$$[(m^{h+1} - 1)/ (m - 1)] (m - 1) = \boldsymbol{m^{h+1} - 1}$$

- When $m = 5$ and $h = 2$ this gives $5^3 - 1 = 124$

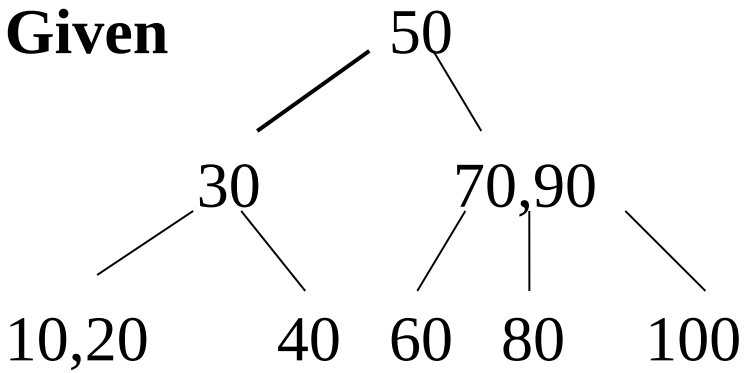
Comparing Trees

- Binary trees
  - Can become *unbalanced* and *lose* their good time complexity (big O)
  - AVL trees are strict binary trees that *overcome the balance problem*
- Multi-way trees
  - B-Trees can be *m*-way, they can have any (odd) number of children
  - One B-Tree, the 2-3 (or 3-way) B-Tree, *approximates* a permanently balanced binary tree, exchanging the AVL tree's balancing operations for insertion and (more complex) deletion operations

2-3 Trees (BTree of order 3)

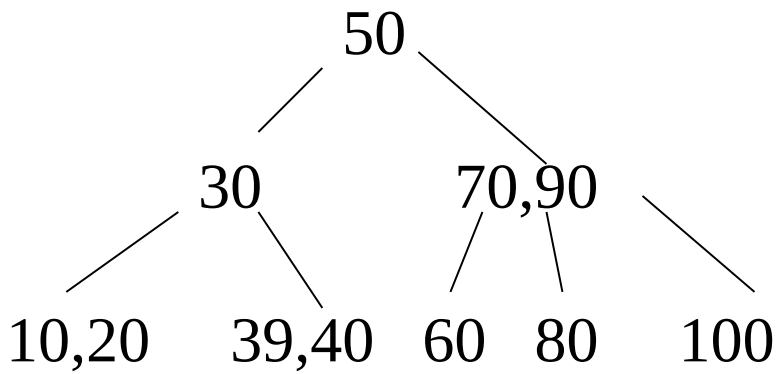
Insertion

Given

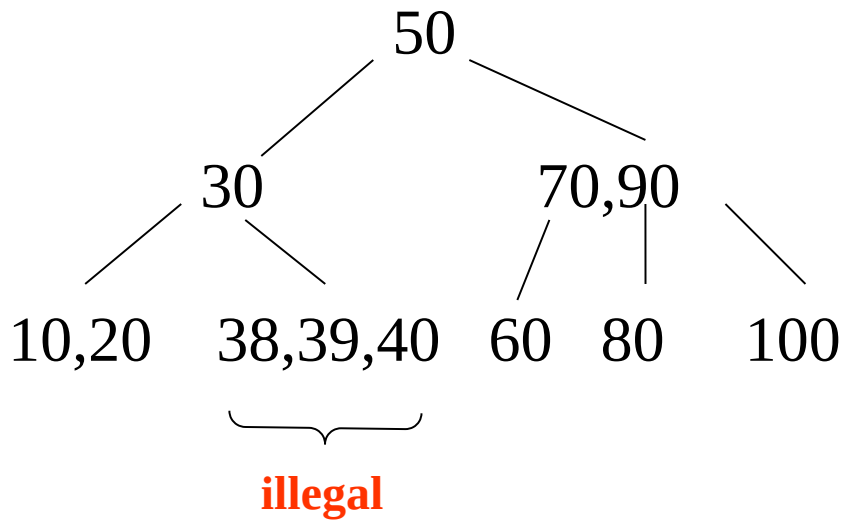


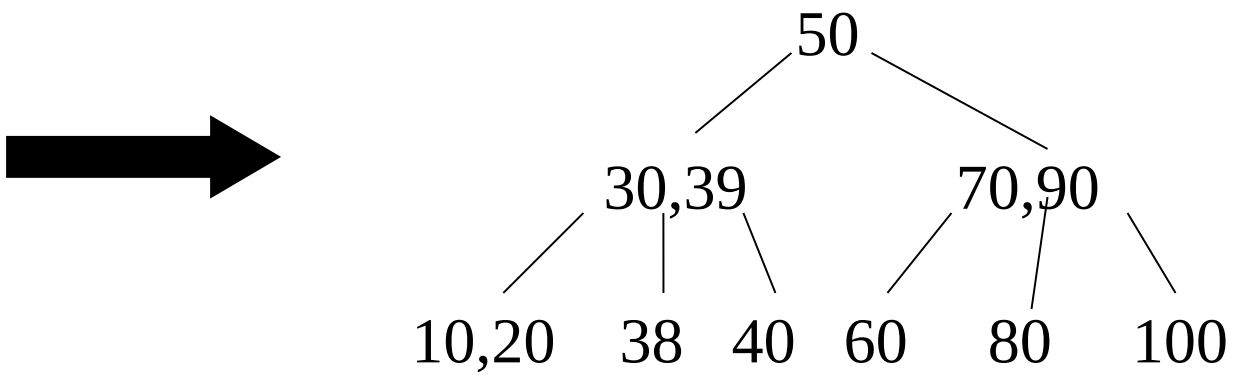
Insertions are always at a leaf

Insert 39

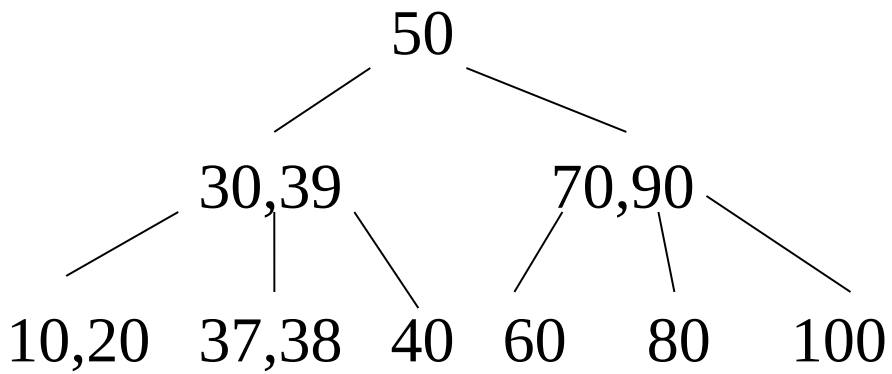


Insert 38


50
30
70,90
10,20
38,39,40
60
80
100
illegal

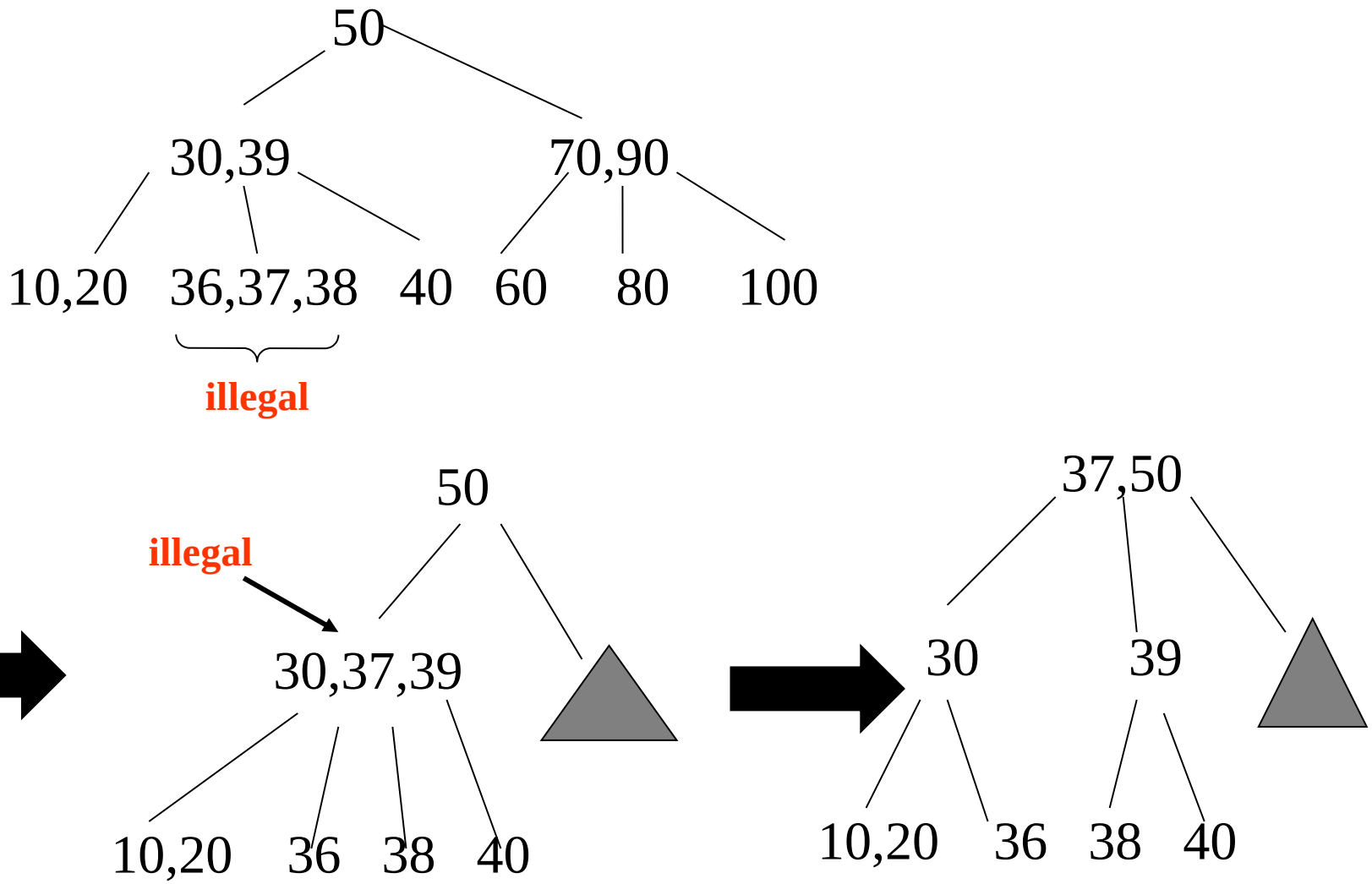

50
30,39
70,90
10,20
38
40
60
80
100

Insert 37

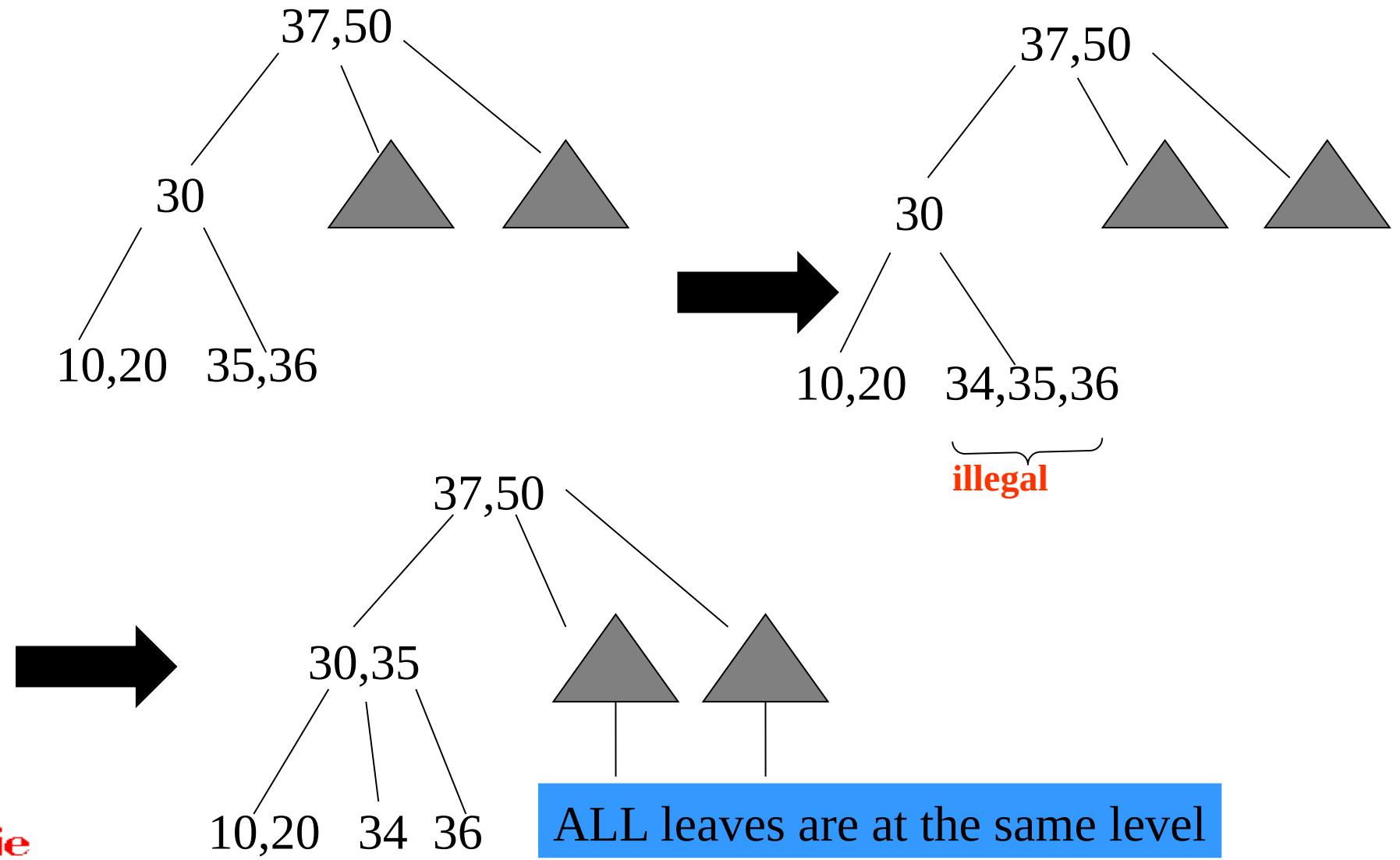


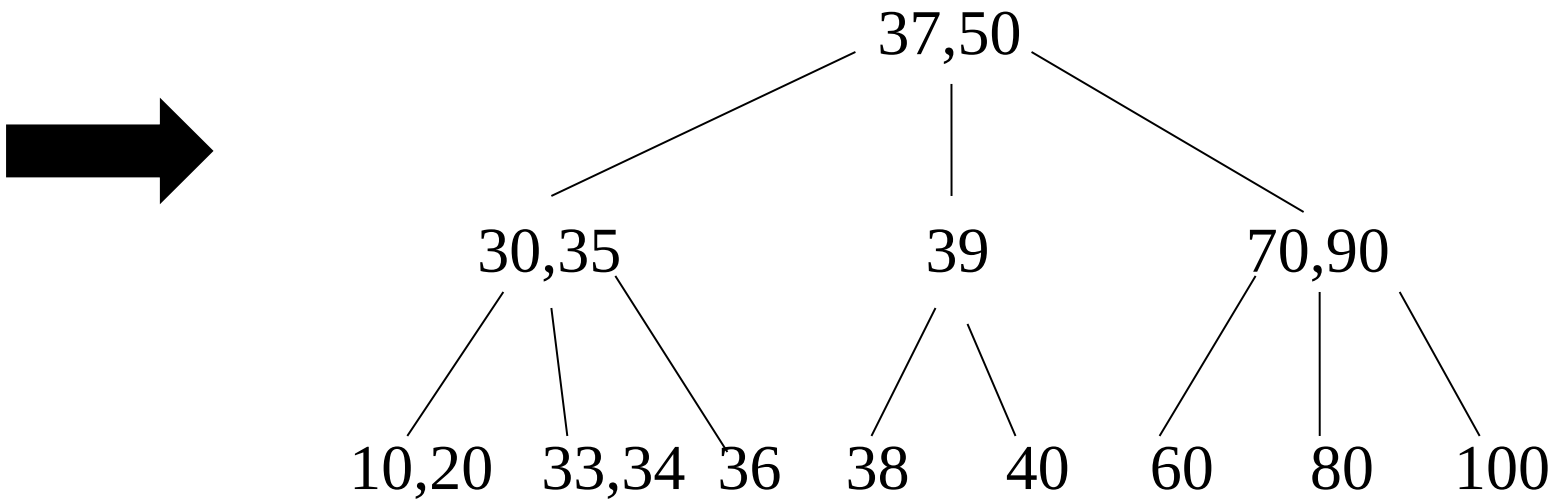
When the height grows it does so from the top.

Insert 36


50
30,39
70,90
10,20
36,37,38
40
60
80
100
illegal
50
illegal
30,37,39
10,20
36
38
40
37,50
30
39
10,20
36
38
40

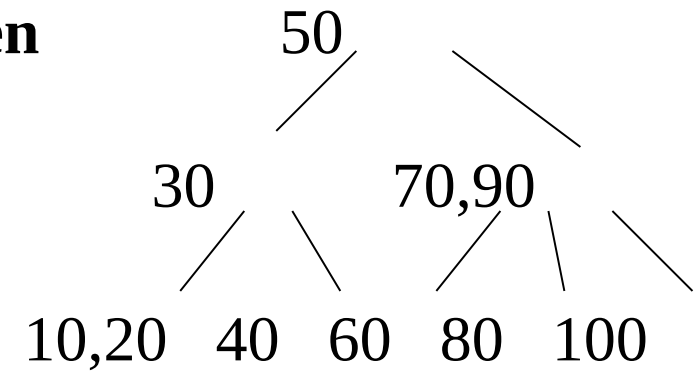
Insert 35, 34, 33



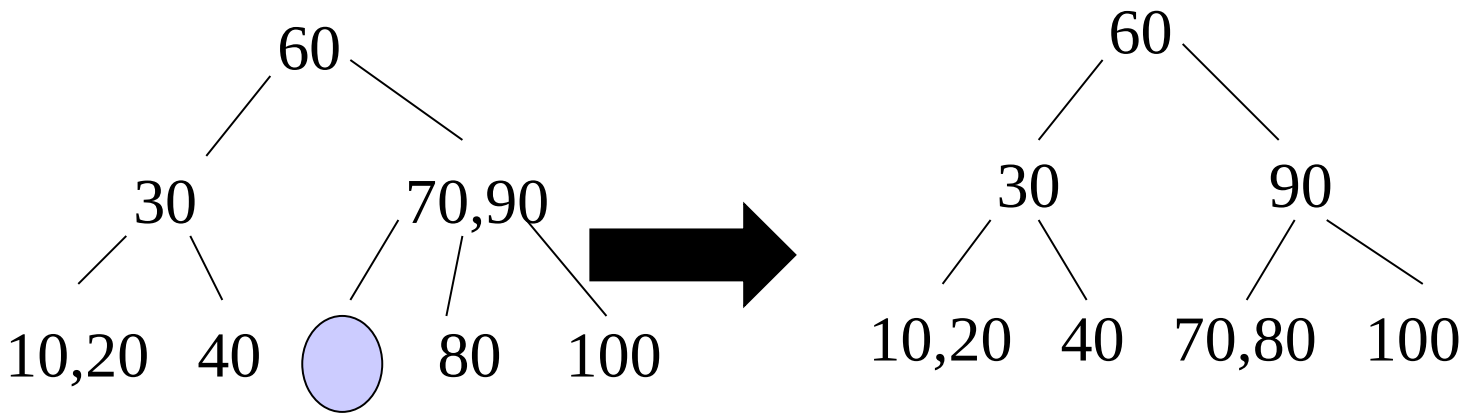

37,50
30,35
39
70,90
10,20
33,34
36
38
40
60
80
100

Deletion

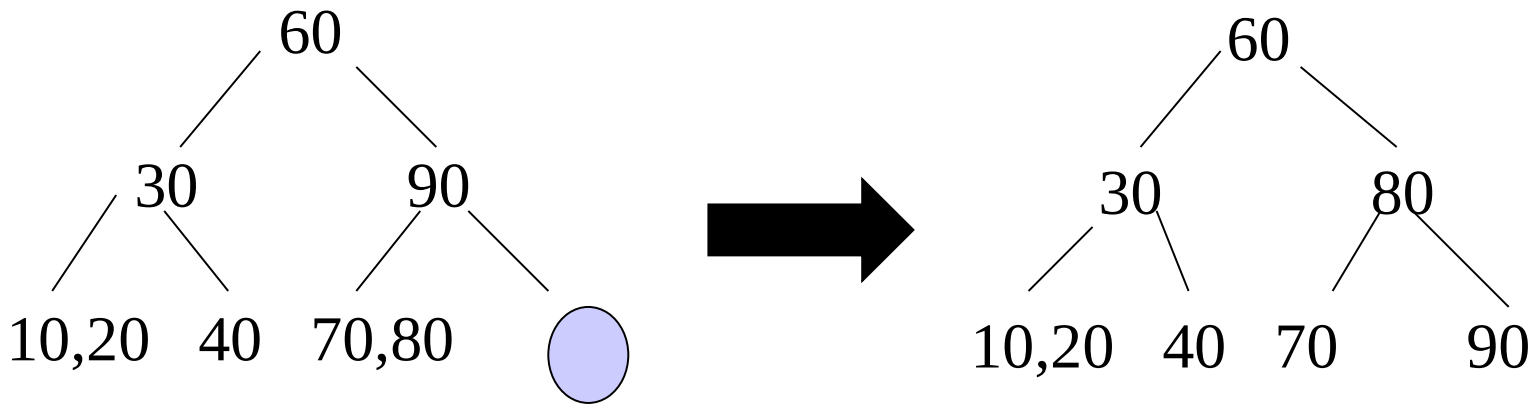
Given



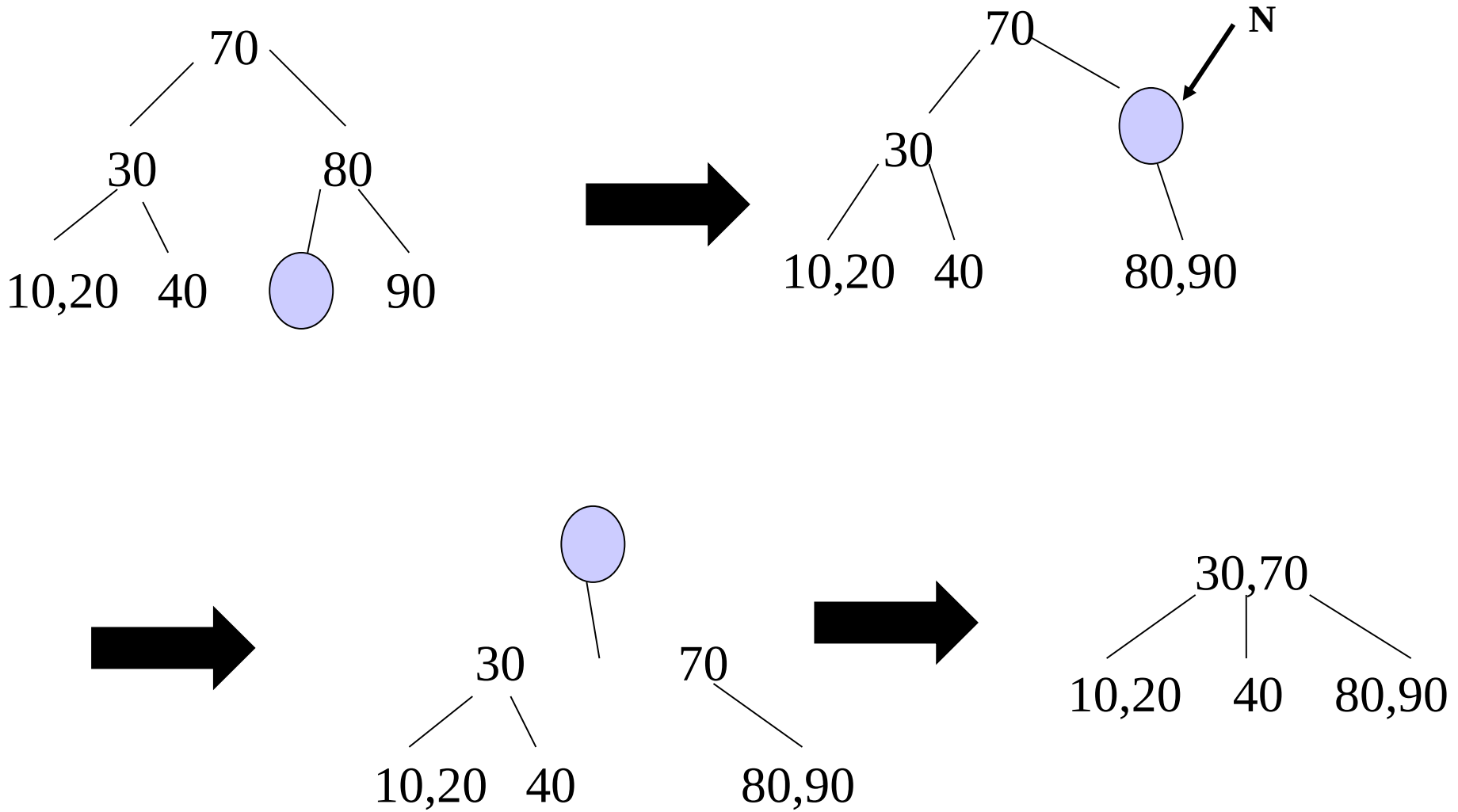
Delete 50



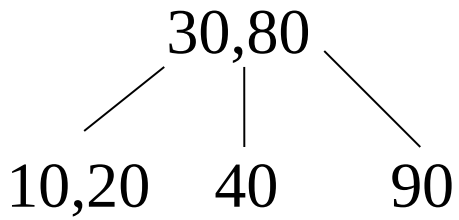
Delete 100



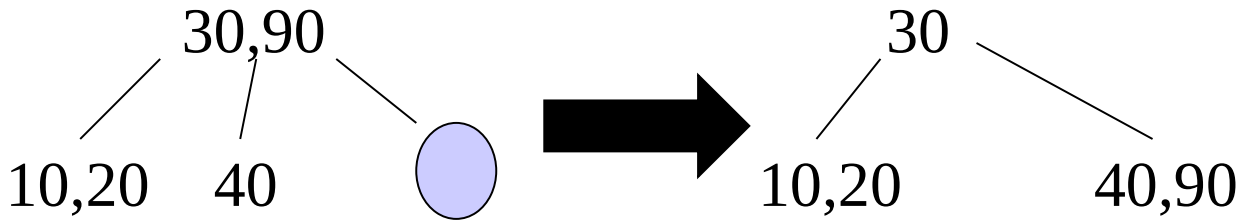
Delete 60


70
30
80
10,20
40
90
70
N
30
10,20
40
80,90
30
70
10,20
40
80,90
30,70
10,20
40
80,90

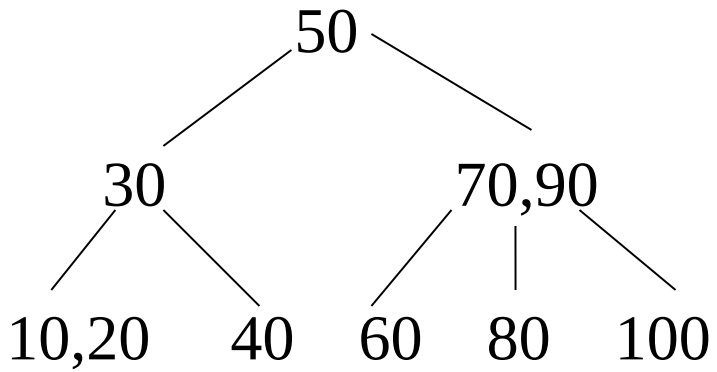
Delete 70



Delete 80

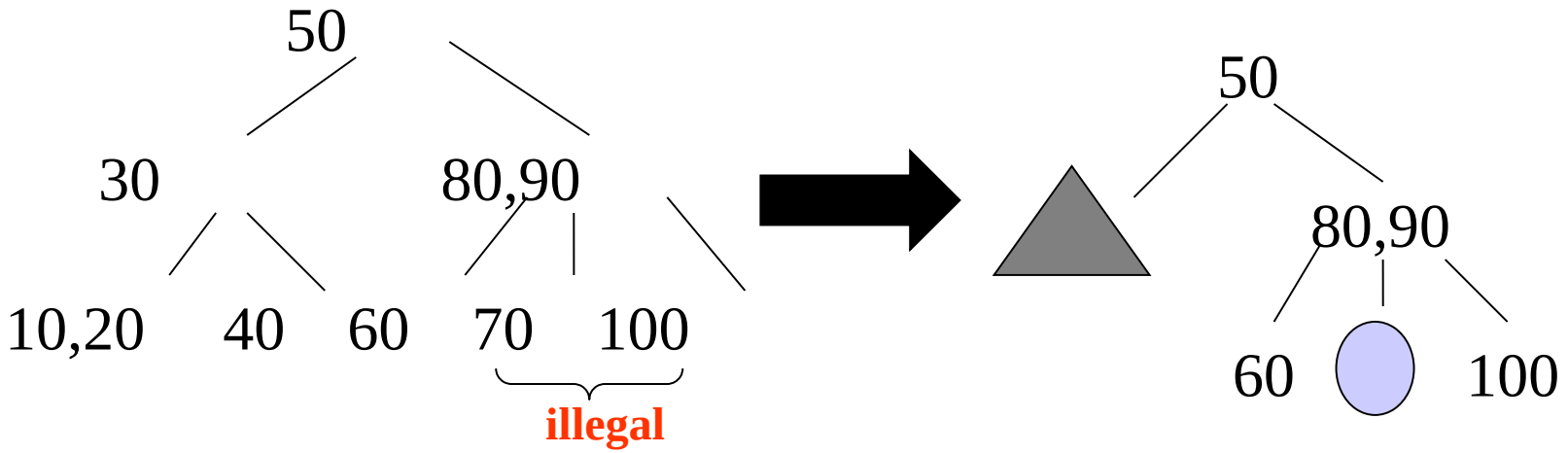


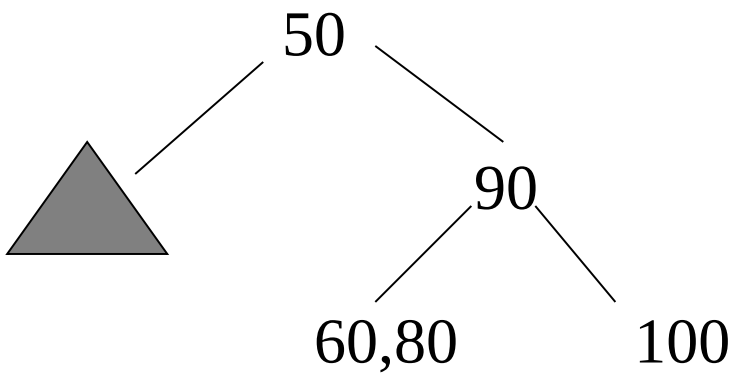
Given

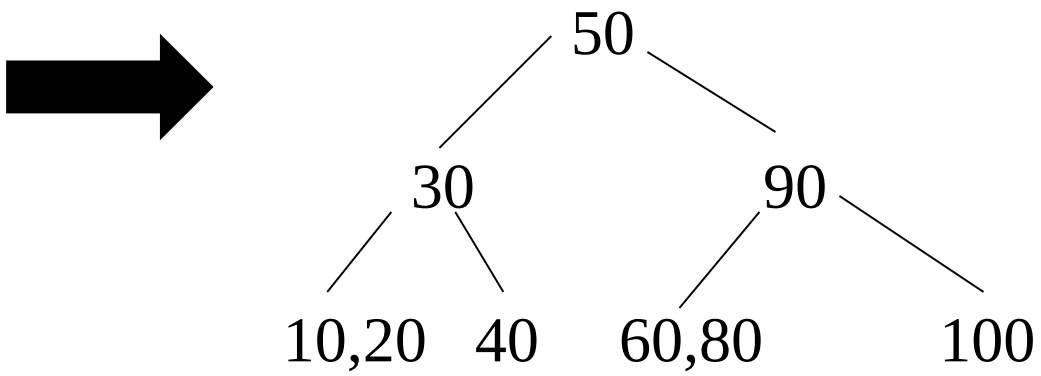


Delete 70

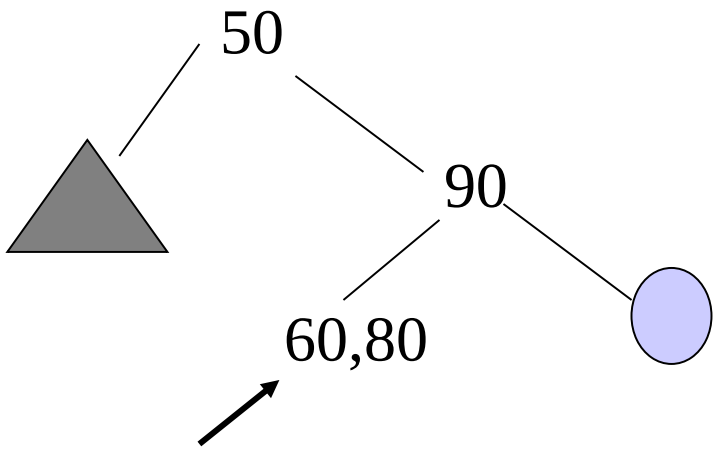
You always begin deletion from a leaf so swap with inorder successor.



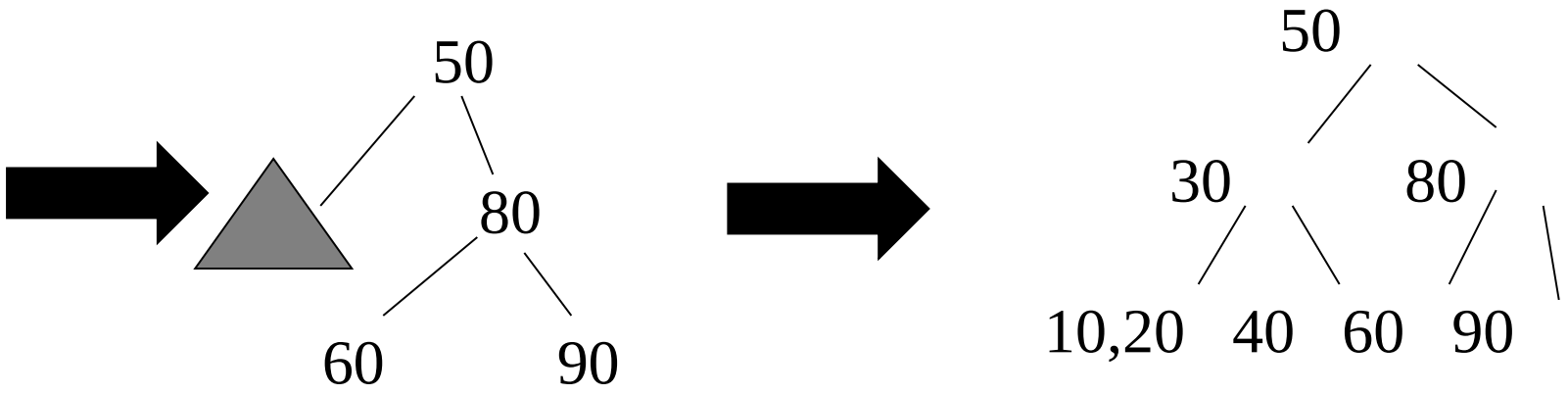

50
90
60,80
100


50
30
90
10,20
40
60,80
100

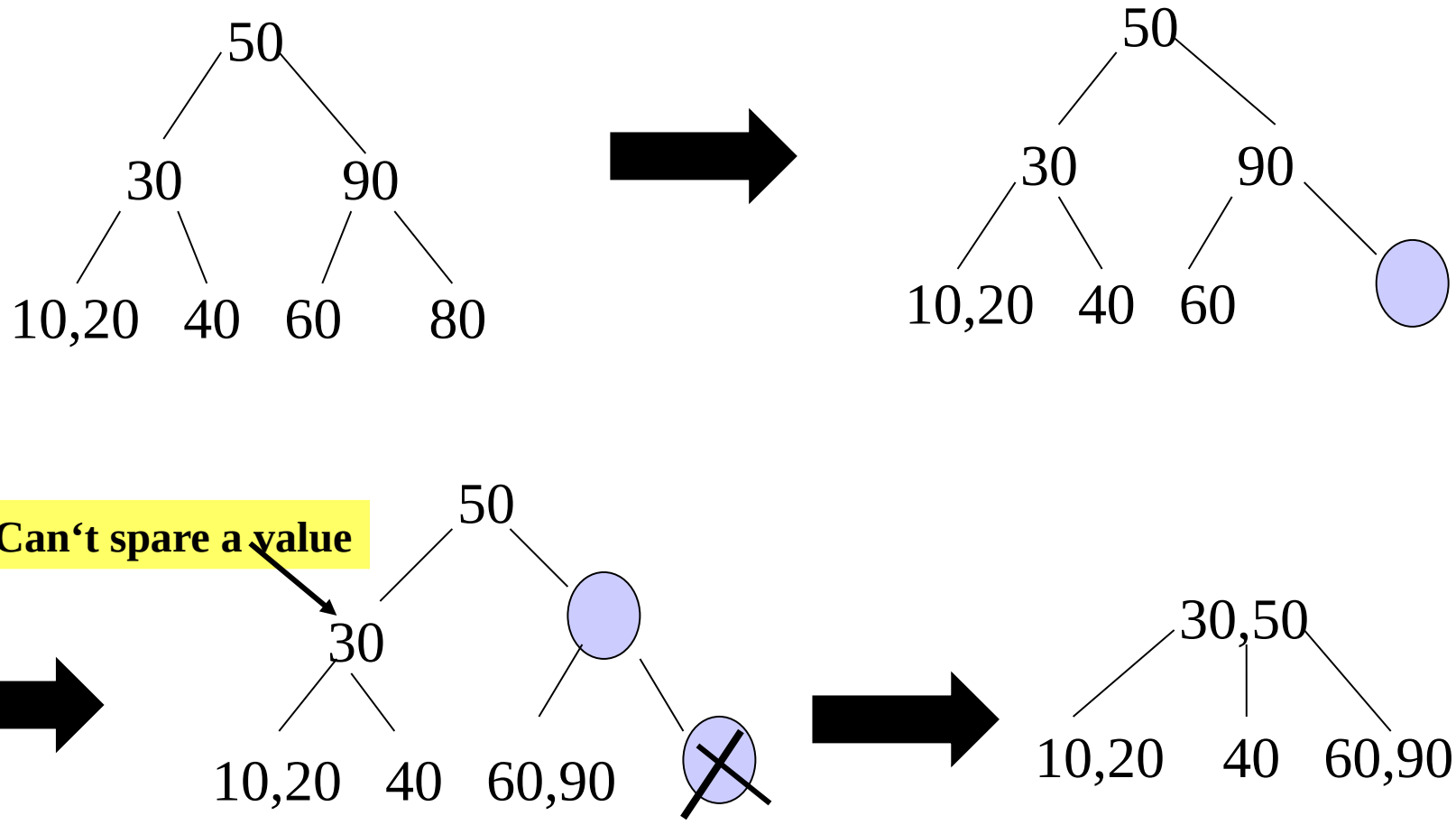
Delete 100



This leaf can spare a value



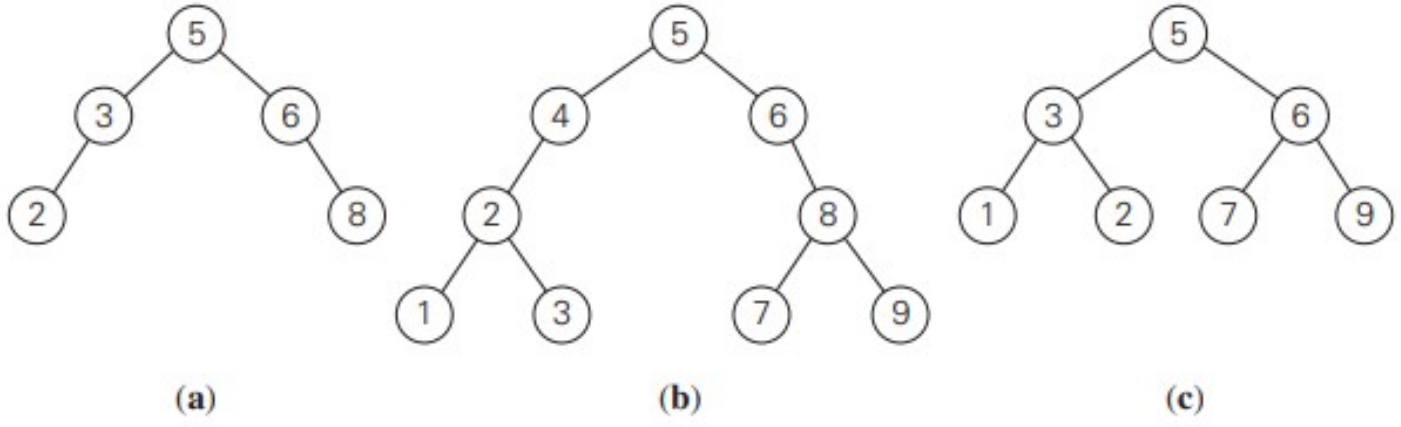
Delete 80


50
30
90
10,20
40
60
80
50
30
90
10,20
40
60
Can‘t spare a value
50
30
10,20
40
60,90
30,50
10,20
40
60,90

https://www.cs.usfca.edu/~galles/visualization/BTree.html

Balanced Search Trees - TRY

1. Which of the following binary trees are AVL trees?



2.a. For n = 1, 2, 3, 4, and 5, draw all the binary trees with n nodes that satisfy the balance requirement of AVL tree.

b. Draw a binary tree of height 4 that can be an AVL tree and has the smallest number of nodes among all such trees.

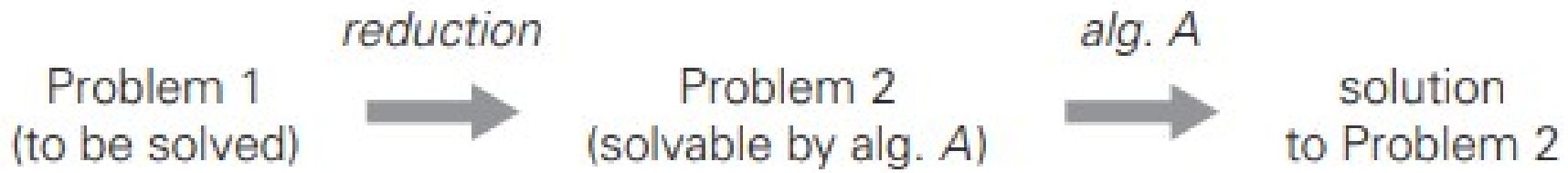
3. a. Construct a 2-3 tree for the list C, O, M, P, U, T, I, N, G. Use the alphabetical order of the letters and insert them successively starting with the empty tree.

b. Assuming that the probabilities of searching for each of the keys (i.e., the letters) are the same, find the largest number and the average number of key comparisons for successful searches in this tree.

Heap and Heap Sort : Refer another ppt

Problem Reduction

- If you need to solve a problem, reduce it to another problem that you know how to solve .



Problem reduction strategy.

Problem Reduction

Examples:

- Computing the least common multiple
- Counting paths in a graph
- Reduction of maximization problems: Maximization and minimization problem.
- Reduction in graph problems.

Computing the Least Common Multiple

- Recall that the least common multiple of two positive integers m and n, denoted lcm(m, n), is defined as the smallest integer that is divisible by both m and n.
- lcm(24, 60) = 120, and lcm(11, 5) =55.

$$24 = 2 \cdot 2 \cdot 2 \cdot 3$$
$$60 = 2 \cdot 2 \cdot 3 \cdot 5$$
$$\text{lcm}(24, 60) = (2 \cdot 2 \cdot 3) \cdot 2 \cdot 5 = 120.$$

Calculate the least common multiple (LCM) of two numbers using the formula:

$$\operatorname{lcm}(m, n) = \frac{m \cdot n}{\gcd(m, n)},$$

where $\gcd(m, n)$ can be computed very efficiently by Euclid's algorithm.

Example:

First, we need to find the greatest common divisor (GCD) of *m* and *n*. We can use Euclid's algorithm for this. LCM(24,60)?

1. The GCD of 24 and 60 is 12.
2. Now, plug the values into the formula for LCM:
   - LCM(24,60)=(24×60)/12 = 120

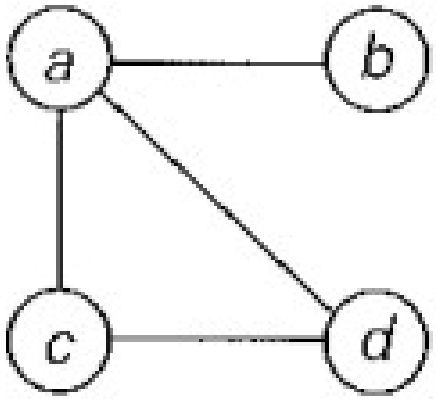
Counting paths in a graph

- Problem: Counting paths between two vertices in a graph.

- It is not difficult to prove by mathematical induction that the number of different paths of length k > 0 from the ith vertex to the jth vertex of a graph (undirected or directed) equals the (i, j)th element of A^k where A is the adjacency matrix of the graph.

- Thus, the problem of counting a graph's paths can be solved with an algorithm for computing an appropriate power of its adjacency matrix.

Counting paths in a gra A graph, its adjacency matrix A, and its square A^2. The elements of A and A^2 indicate the number of paths of lengths 1 and 2, respectively.



$$A = \begin{array}{c} \\ a \\ b \\ c \\ d \end{array} \begin{array}{c} \begin{array}{cccc} a & b & c & d \end{array} \\ \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix} \end{array} \qquad A^2 = \begin{array}{c} \\ a \\ b \\ c \\ d \end{array} \begin{array}{c} \begin{array}{cccc} a & b & c & d \end{array} \\ \begin{bmatrix} 3 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 1 & 1 & 2 & 1 \\ 1 & 1 & 1 & 2 \end{bmatrix} \end{array}$$

- Its adjacency matrix A and its square A^2 indicate the number of paths of length 1 and 2, respectively, between the corresponding vertices of the graph.
- In particular, there are three paths of length 2 that start and end at vertex a: a - b- a, a - c- a, and a - d -a, but only one path of length 2 from a to c: a - d -c.

Reduction to Graph Problems

- Using State space graph.
- States: Nodes or vertices in the graph represent individual states or configurations of a system. Each state captures relevant information about the problem being modeled.
- Transitions: Directed edges or arcs between states represent possible transitions or actions that can be taken to move from one state to another. These transitions typically correspond to allowable moves, decisions, or events in the problem domain.
- Initial State: One of the states in the graph is designated as the initial state, representing the starting point of the system or problem.
- Goal State(s): One or more states in the graph are designated as goal states, representing the desired outcomes or solutions of the problem.